

USENIX

THE ADVANCED COMPUTING SYSTEMS ASSOCIATION

Remise: Authorized Anonymous Communication Systems

Rohan Ravi, Paritosh Shukla, and Adithya Vadapalli, *IIT Kanpur*

https://www.usenix.org/conference/usenixsecurity26/presentation/ravi-rohan

This paper is included in the Proceedings of the 35th USENIX Security Symposium.

August 12–14, 2026 • Baltimore, MD, USA

ISBN 978-1-939133-58-8

Open access to the Proceedings of the 35th USENIX Security Symposium is sponsored by



جامعة الملك عبدالله للعلوم والتقنية

King Abdullah University of Science and Technology

ARTIFACT EVALUATED
USENIX
AVAILABLE

Remise: Authorized Anonymous Communication Systems

Rohan Ravi
IIT Kanpur
rohanra@cse.iitk.ac.in

Paritosh Shukla
IIT Kanpur
paritoshs24@cse.iitk.ac.in

Adithya Vadapalli
IIT Kanpur
avadapalli@cse.iitk.ac.in

Abstract

We present Remise, a two-server authorized anonymous communication system built on Distributed Oblivious RAM (DORAM). Remise supports two modes of operation: (i) an anonymous bulletin board, where messages are publicly revealed at the end of each epoch without linking senders to messages, and (ii) anonymous communication channels, where messages remain secret-shared, and writers selectively grant and revoke read access to chosen readers. In both modes, the two servers execute read and write operations without learning which indices are accessed, which authorization tokens are used, or which relationships exist between writers and readers, assuming no collusion between the servers. A central contribution of Remise is a lightweight and efficient access-control mechanism. Authorization tokens are maintained in secret-shared form across the two servers, enabling oblivious verification and updates, while preventing leakage even under client–server collusion. Unlike prior DPF-based systems, Remise provides built-in auditing by having the semi-honest servers generate standard-basis vector shares internally, eliminating the need for server-side DPF validity checks. We implement a prototype of Remise and evaluate it under realistic network conditions. Our experiments show an 80× improvement in online server time for request validity check when compared to PACL (IEEE S&P 2023) for large database size.

1 Introduction

The birth of the Internet was accompanied by great optimism. It was envisioned as a democratizing force—one that would enable free exchange of ideas, universal access to information, and participation independent of geography or social status. While this vision has been realized in many respects, the modern Internet also exposes users to pervasive surveillance, manipulation, and censorship [19, 23]. As a result, users increasingly feel compelled to self-censor and communicate with caution. In this context, *anonymous communication systems* offer a compelling response. By decoupling identities from messages, such systems protect users from surveillance and retaliation and play a critical role in preserving free expression. They are particularly valuable for journalists, whistleblowers, activists, and ordinary users operating in adversarial environments.

1.1 Anonymous Communication Systems

Sasy and Goldberg [27] classify anonymous communication systems into (i) sender-unlinkable broadcast systems (SUBS), (ii) sender-unlinkable messaging systems (SUMS), (iii) relationship-unobservable systems (RUS), and (iv) communication-unobservable systems (CUS). In this work, we focus on the first three categories. In a SUBS, users anonymously post messages to a public bulletin board that is typically secret-shared across multiple servers and reconstructed at fixed epochs. While the set of messages and the anonymity set of broadcasters are public, the link between a message and its author is hidden; cover traffic further prevents distinguishing real messages from dummy ones. Conceptually, SUBS resemble an anonymous microblogging platform. In a SUMS, receivers own mailboxes identified by secret mailbox IDs that are shared out of band with intended senders. Senders remain anonymous, only authorized senders can write to a mailbox, and receivers obtain only messages addressed to them, though receiver anonymity is not provided. This model is commonly used for confidential disclosures, such as journalist–source communication (e.g., SecureDrop [14]). RUS extend SUMS by additionally ensuring that even a global adversary cannot link senders to receivers.

Sender Anonymous Communication Channels: This work introduces a new type of RUS that has not been studied hitherto, namely, Sender Anonymous Communication

Channels. In this model, a set of senders posts messages on a secret message board shared by two parties. However, the parties never reconstruct the messages to post them to the public. Instead, writers give read access to readers of their choice. Furthermore, the writers can revoke the access whenever they wish to do so. We could also think about this as an anonymous *Substack* or *blog*. The subscribers (of the *Substack* or *Blog*) are the readers who have been granted read access by the authors. The authors (the writers) are granted write access by the service provider (the Remise servers). The writers have "ownership" of that space in the sense that they can write messages of their choice in the space, allow only the people of their choice to read messages from that space, and can remove read access for any reader at any point.

1.2 Review of State-of-the-Art

Remise follows the lineage of Anonymous Communication Systems like Riposte, Express, Sabre, and Spectrum, which are based on the so-called Distributed Point Functions (DPFs). We will not go into the details of DPFs, however, it suffices to regard a DPF as a functional, succinct representation of secret shares of a 1-hot vector. The servers use shares of a 1-hot vector produced by DPFs to perform read and write operations on a database without revealing which element was accessed. Such systems typically need to perform two checks before allowing a write request to be executed: (i) the DPFs are valid, i.e., they will expand to the shares of a 1-hot vector *(audit)*, and (ii) the user is allowed to access at the index at which they are attempting to do so *(authorization)*.

Riposte: Corrigan-Gibbs, Boneh, and Mazières, presented Riposte [7], a sender-anonymous broadcasting system that works in the bulletin board model. In Riposte, the writers (or broadcasters) submit write requests, creating a DPF at a random location and sending it to the servers. The servers reconstruct and publish all the messages that were received during each epoch. A problem with users writing at random locations on the bulletin board is that a broadcaster could overwrite a message written by another broadcaster. To reduce the probability of such a write-collision to 5%, the size of the Riposte servers is 19.5 times the number of messages to be posted. Riposte runs an audit protocol to ascertain that the write requests submitted are well-formed DPFs. One major weakness of Riposte is that it uses $O(\sqrt{n})$-sized DPFs, where n is the size of the database. This is because the structure of these DPFs make their auditing protocol efficient.

Express: In Express [13], each receiver is assigned a mailbox together with a $\lambda = 128$-bit mailbox ID by the servers. Receivers share this mailbox ID with intended senders, who write messages by generating DPFs targeting the corresponding ID. Although senders generate DPFs over a domain of size 2^{128}, the Express servers evaluate them only at registered mailbox IDs. Express improves upon Riposte by using logarithmic-sized DPFs together with SNIPs for auditing. However, auditing requires the servers to evaluate each submitted DPF at all registered mailbox IDs, incurring work linear in the number of mailboxes. As a result, Express is vulnerable to DoS attacks in which malicious clients submit malformed DPFs that force expensive evaluations before rejection. Moreover, Express is not secure against server–client collusion, since servers know all registered mailbox IDs and can thereby derive and leak write authorization for any mailbox.

Sabre: The work by Vadapalli, Storrier, and Henry, Sabre [31] extends the work by Express and prevents DoS attacks by presenting $O(\lg n)$ auditing. For authorization, Sabre uses a mailbox ID verification using pseudorandom functions. If a writer is authorized to write at the $\mathtt{i}^{*\text{th}}$ location, the writer should know $\mathtt{PRF}(\mathtt{k},\mathtt{i}^*)$ while only the servers know the secret key, $\mathtt{k}$. The writer submits the additive shares of $\mathtt{PRF}(\mathtt{k},\mathtt{i}^*)$ along with the shares of $\mathtt{i}^*$ to the Sabre servers. Then, the Sabre servers use secure multiparty computation(MPC) to compute $\mathtt{PRF}(\mathtt{k},\mathtt{i}^*)$, and check for equality. However, the MPC computation requires the underlying cipher to have a low multiplicative depth. Like Express, Sabre is also not secure when a collusion between a server and a user occurs, as the servers know the authorization proof, $\mathtt{PRF}(\mathtt{k},\mathtt{i})$, for all index $\mathtt{i}$, by virtue of knowing $\mathtt{k}$.

Spectrum: Spectrum [22] is an anonymous broadcasting system introduced by Newman, Servan-Schreiber, and Devadas with a low number of broadcasters. Each broadcaster in Spectrum is allotted a "channel" where they are authorized to write. This prevents write-collisions. Spectrum presents a blind access control mechanism that allows the servers to perform auditing and authorization over the DPFs. In addition, Spectrum has a Blame-Game protocol that protects against malicious Spectrum servers from rejecting valid write requests. Unlike Express and Sabre, Spectrum is secure against server-writer collusion leading to leakage of authorization material under the discrete log hardness assumption.

PACL: Unlike prior work on anonymous communication, PACL [28] addresses access control for Function Secret Sharing, which can also be applied to DPF-based communication systems. It builds on the Spectrum method for blind access control and introduces more efficient techniques. In the basic setup, servers hold a vector of verification keys $\Lambda = \{g^{\alpha_1}, \ldots, g^{\alpha_n}\}$, where secret key $\alpha_{\mathtt{i}}$ authorizes access to index $\mathtt{i}$ (DPF generation at $\mathtt{i}$). The DPF is used to locally select (multiplicative) secret shares of $g^{[\alpha_{\mathtt{i}}]}$,enabling verification through addi-

Table 1: Comparison of Remise with prior systems. N = database size, L = number of registered writers, λ = security parameter. "Auth Secret Hiding" indicates whether per-index authorization material is hidden from individual servers. "Auth Update Privacy" indicates whether authorization state can be updated without server visibility. For Riposte, $N > L$; for other systems $N = L$. Riposte does not provide authorization. * Authorization overlaps with database write. † Audit and authorization are unified. ‡ Clients send secret shares of index instead of DPF keys, requiring $\log L$ bits.

System	Client Req. Size	Server Audit Work	Server Auth Work	Auth Secret Hiding	Auth Update Privacy
Riposte	$O(\sqrt{N})$	$O(N)$	-	-	-
Express	$O(\lambda^2)$	$O(L)$	$O(L)^\star$	×	×
Sabre	$O(\log L)$	$O(\log L)$	$O(1)$	×	×
Spectrum	$O(\log L)$	$O(L)$	$O(L)^\dagger$	✓ (DL-Hard)	×
Spectrum + PACL	$O(\log L)$	$O(L)$	$O(L)^\dagger$	✓ (DL-Hard)	×
Remise	$O(\log L)^\ddagger$	-	$O(L)$	✓ (Secret-shared)	✓

tive shares of a proof $[\Pi]_k \leftarrow [-\alpha_i]_k$. The servers then check that $g^{[\alpha_i]_k} \cdot g^{[\Pi]_k}$ yields additive shares of 1. This essentially generalizes the Spectrum protocol to multiple channels, allowing simultaneous auditing and authorization. PACL improves efficiency by replacing exponent-based checks with a *Schnorr proof over secret shares*, which proves knowledge of the discrete logarithm of an additively shared element. A second construction, the *inclusion predicate*, supports multiple access keys per index. Here, servers store an $n \times \ell$ matrix of verification keys and use one DPF to select the ith row and another to pick the correct column via correction terms, yielding the required shares for the audit.

We compare Remise and prior systems in Table 1. We only consider variants that use 2 main servers (for example, Spectrum can be instantiated with more than 2 servers).

1.3 System Overview

Remise[1] is a two-server anonymous communication system that enforces authorization for both read and write operations while providing anonymity. The servers verify authorization without learning the accessed index, the message or the authorization proof. The system supports two instantiations: a public bulletin board model (RemiseBB) and a communication channels model (RemiseCC). In RemiseBB, authorized writers anonymously post messages to pre-assigned slots in the bulletin board. This bulletin board is periodically reconstructed and published. All write requests made by writers (actual or cover traffic), look indistinguishable to the servers or an adversary observing the network. As a result, the link between the writers and the published messages is severed. In RemiseCC, the messages are posted by writers similarly to RemiseBB, except that they are not published. Instead, writers can selectively grant and revoke read access to chosen readers and only an authorized reader can read from a given index. RemiseCC allows writers to update (grant or revoke) read permissions. The RemiseCC servers remain oblivious to both the identities of authorized readers and any changes in the authorization state. Here, all read and write requests (actual and cover traffic) are indistinguishable to the servers and the network, which hides writer-reader link.

Figure 1 gives a brief overview of RemiseBB and RemiseCC and its interaction with users.

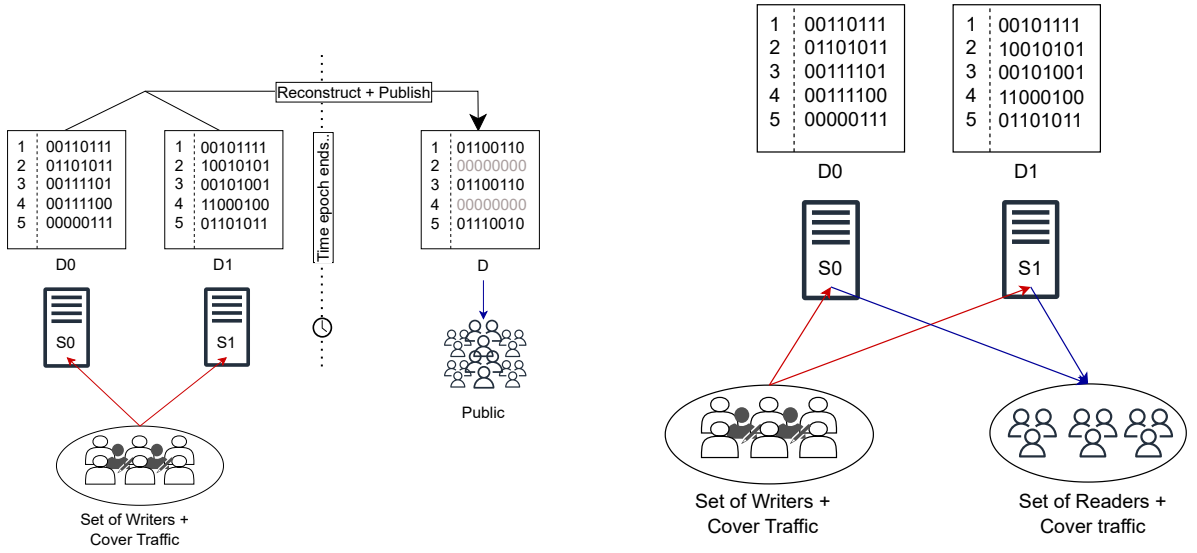
Contributions We make the following contributions: (i) We introduce a DORAM-based authorization mechanism for message boards that provides built-in auditability, (ii) We design communication channels in which writers can grant read permissions to readers without servers learning which reader received access, and (iii) We support revocation of access privileges while hiding from the servers whose permissions were modified.

1.3.1 Main ideas in realizing Remise

Distributed ORAM for hidden accesses: Remise is a Distributed Oblivious Random Access Memory (DORAM) based Anonymous Communication System. While we defer the detailed discussion of DORAMs to Section 2.3, it suffices for now to know that DORAMs enable multiple parties, each holding secret shares of a memory and secret shares of an index, to perform an oblivious (write or read) access to the memory at the index. Remise servers act like DORAM parties and perform operations on the secret shared database and secret shared inputs.

Built-in Auditing: In previous systems that use DPFs [7, 13, 22, 31] to perform an access on the secret shared databases held by the servers, the client provides the DPF keys for the servers to evaluate. Consequently, the servers have to perform an *audit* check to ensure that

[1] Full version available at [25]



(a) RemiseBB: Remise instantiated in the anonymous bulletin board model. Requests from writers and cover traffic are indistinguishable to servers and any adversary. Writers write to allocated locations, which the servers reconstruct and publish at the end of each epoch.

(b) RemiseCC: Remise instantiated in the anonymous communication channels model. Authorized writers post messages accessible only to authorized readers with message content hidden from all other entities. Write requests (including cover traffic) are indistinguishable from one another, and so are read requests, while writer-reader links remain hidden.

Figure 1: Remise system overview

the DPF keys evaluate to valid shares of a 1-hot vector, otherwise malformed DPFs could corrupt the entire database. A consequence of using DORAMs (like Floram [11] and Duoram [30], which Remise does) is that the shares of the standard-basis vector are generated by the servers themselves, thereby offloading this computation from potentially malicious clients to the servers.

Resistance to Client–Server Collusion: In Remise, authorization tokens are stored as secret shares across servers, ensuring that authorization secrets remain hidden under client–server collusion, as long as at least one server is semi-honest. In contrast, Express and Sabre store authorization material in the clear, allowing a colluding server to leak authorization secrets. While Spectrum (and PACL) prevent this leakage under computational assumptions, Remise provides information-theoretic security.

Authorization Revocability Remise (as RemiseCC) supports authorization revocability by allowing writers to obliviously update read authorization tokens (proofs) associated with messages they authored. This functionality is essential in anonymous communication settings where a writer controls read access for a dynamic set of readers. Writers may grant access to new readers or revoke access from previously authorized readers, while preserving unlinkability between writers, readers, and database indices. As a consequence of authorization tokens being maintained in secret-shared form, Remise enables these updates to be performed via oblivious DORAM update operations, without exposing which authorization state was modified. In contrast, while Spectrum (and PACL) hide the secret key used for authorization, any modification to the vector of verification keys is directly visible to the servers, precluding oblivious authorization updates.

2 Background

2.1 Secret Shares

Secret sharing is a cryptographic mechanism for distributing a secret among multiple parties such that any subset of fewer than t parties learns no information about the secret, while any subset of t or more parties can reconstruct it. The parameter t is called the *threshold* of the secret-sharing scheme. Secret-sharing schemes come in many flavors. We denote additive secret sharing of a value x modulo 2^r by x^{AS}. When using (k,k) additive sharing among k parties, the individual shares are denoted by $x_0^{\mathrm{AS}},\ldots,x_{k-1}^{\mathrm{AS}}$, and satisfy $\sum_{i=0}^{k-1} x_i^{\mathrm{AS}} \equiv x \pmod{2^r}$. We use b^{BS} to denote a Boolean share, and

x^{GS} to denote a generic (secret) sharing of x. An alternative way to secret-share an index $\mathtt{i}^*$ is via shares of a *one-hot vector*, i.e., a vector that is zero everywhere except at the position $\mathtt{i}^*$. Such shares are particularly useful in sender-anonymous communication. Distributed Point Functions (DPFs) [4] provide a succinct mechanism for secret-sharing standard-basis vectors. In particular, $(2,2)$-DPFs allow two parties to share a standard-basis vector and serve as a core building block of the Distributed ORAM constructions used in Remise.

2.2 Secure Multiparty Computation

Secure Multiparty Computation (MPC) is a cryptographic technique where parties $\mathtt{S}_1,\ldots,\mathtt{S}_n$ holding private inputs $x_1,\ldots,x_n$ jointly compute a function $f(x_1,\ldots,x_n)$ such that no party learns anything beyond what can be learned from the output. There are three main metrics to evaluate the performance of an MPC protocol, namely, (i) *local computation*, the computational work performed by each party, (ii) *bandwidth*, the total communication exchanged between parties during the protocol, and (iii) *round complexity*, the number of communication rounds (network latencies) required.

In the following we will present some of the MPC protocols that will be used in the paper.

Multiplication: We denote the multiplication operation as $z^{\mathrm{GS}} \leftarrow \textsc{Mult}(x^{\mathrm{GS}}, y^{\mathrm{GS}})$. Du-Atallah [12] is a variant of the classic Beaver's method [3] to do multiplication on secret shares, which we use in this paper. While Du-Atallah is a $(2+1)$-Party protocol, it can be easily converted to a 2P-Protocol by using OT (Oblivious Transfer) to produce correlated randomness. We model the protocol using the ideal functionality $\mathcal{F}_{\mathsf{MULT}}$.

EQZ Protocol. The *Equal-to-Zero* protocol checks whether a secret-shared value is equal to zero. We denote the operation by $b^{\mathrm{AS}} \leftarrow \textsc{Eqz}(x^{\mathrm{AS}})$, where the parties holding additive shares of x obtain shares of a bit b such that $b=1$ iff $x=0$, and $b=0$ otherwise. We model the protocol using the ideal functionality $\mathcal{F}_{\mathsf{EQZ}}$.

2.3 Oblivious Memory Access

Distributed Oblivious Read-Only Memory (DOROM) supports oblivious reads from a database that is fixed after setup. Let $\mathrm{D} \in \mathbb{F}^n$ be a database and let $\mathtt{i}^* \in [n]$ denote an index. The two servers $\mathtt{S}_0$ and $\mathtt{S}_1$ hold an encrypted representation of the database such that for every $\mathtt{i} \in [n]$, $\mathrm{D}[\mathtt{i}] + \mathtt{PRF}(\mathtt{k}_0,\mathtt{i}) + \mathtt{PRF}(\mathtt{k}_1,\mathtt{i})$ is stored by the servers, while $\mathtt{S}_b$ knows only $\mathtt{k}_b$. An oblivious read at a secret-shared index $\mathtt{i}^{*\mathrm{AS}}$ is denoted by $\mathrm{D}^{\mathrm{RO}}[\mathtt{i}^{*\mathrm{AS}}]$ which outputs additive shares of $\mathrm{D}[\mathtt{i}^*]$ while leaking only invocation metadata. A read operation is implemented using PIR over the encrypted database, followed by a lightweight MPC step to remove the PRF masks. We model the protocol using the ideal functionality $\mathcal{F}_{\mathsf{DOROM}}$.

Distributed Oblivious Write-Only Memory (DOWOM) supports oblivious updates to a secret-shared database. The two servers $\mathtt{S}_0$ and $\mathtt{S}_1$ hold additive shares $\mathrm{D}_0^{\mathrm{AS}}, \mathrm{D}_1^{\mathrm{AS}}$ of the database. An oblivious update at a secret-shared index $\mathtt{i}^{*\mathrm{AS}}$ by a secret-shared value $\mathtt{M}^{\mathrm{AS}}$ is denoted by: $\mathrm{D}^{\mathrm{AS}}[\mathtt{i}^{*\mathrm{AS}}] \leftarrow \mathrm{D}^{\mathrm{AS}}[\mathtt{i}^{*\mathrm{AS}}] + \mathtt{M}^{\mathrm{AS}}$. The update modifies only the target location while hiding both the accessed index and updated value. Operationally, the servers locally add additive shares of $\mathbf{e}_{\mathtt{i}^*} \cdot \mathtt{M}$ to their database shares, where $\mathbf{e}_{\mathtt{i}^*}$ is standard basis-vector at $\mathtt{i}^*$. The protocol leaks only invocation metadata. We model the protocol using the ideal functionality $\mathcal{F}_{\mathsf{DOWOM}}$.

Distributed Oblivious RAM (DORAM) supports both oblivious reads and oblivious writes. Formally, given additive shares of a database D^{AS}, an index $\mathtt{i}^{*\mathrm{AS}}$, and an update value $\mathtt{M}^{\mathrm{AS}}$, a DORAM supports: (i) oblivious reads producing shares of $\mathrm{D}[\mathtt{i}^*]$, and (ii) oblivious writes producing updated database shares $\mathrm{D}'^{\mathrm{AS}}$ such that $\mathrm{D}'[\mathtt{i}] = \mathrm{D}[\mathtt{i}]$ for all $\mathtt{i} \neq \mathtt{i}^*$, and $\mathrm{D}'[\mathtt{i}^*] = \mathrm{D}[\mathtt{i}^*] + \mathtt{M}$. The protocol leaks only invocation metadata. We model the protocol using the ideal functionality $\mathcal{F}_{\mathsf{DORAM}}$.

Supporting both operations simultaneously is more expensive than specialized DOROM or DOWOM constructions. This is because the protocol must either switch between distinct memory layouts, as in Floram [11], or maintain additional auxiliary state such as blinds and blind shares, as in Duoram [30]. In this work, we instantiate DOROM and DOWOM using Floram [11], and instantiate full DORAM using 2P-Duoram [30].

Preprocessing in DORAMs All the standard-basis vector shares needed in DORAMs (including read only and write only) are produced using DPFs. It is important to note that Remise servers produce the DPFs in a preprocessing phase (before the client obliviously informs where to write and what to write) and adjust them to the correct index and value in the online phase with constant communication. For more details, we refer the reader to the Duoram paper [30], beginning with §4.1 which describes the underlying $(2+1)$-party protocol. The two-party variant, as further discussed in the paper, is obtained by using Oblivious Transfer to generate multiplicative triples.

3 Authorized Messaging

Remise operates with two non-colluding servers, $\mathtt{S}_0$ and $\mathtt{S}_1$, and can be instantiated in two models: (i) the *Anonymous Bulletin Board* model, denoted as RemiseBB, and (ii) the *Anonymous Communication Channels* model, denoted as RemiseCC. Two main types of users in-

teract with Remise, namely, (i) writers, and (ii) readers. A subset of users may also generate *cover traffic* by issuing dummy read or write requests. For writes, cover traffic consists of zero-message updates, while for reads, it consists of valid read requests to a reserved dummy index. For the Remise servers, such requests are information-theoretically indistinguishable from real access requests and serve the purpose of increasing the size of the anonymity set.

Setup Phase The functioning of Remise (both RemiseBB and RemiseCC) begins with a setup phase, during which writers obtain authorization to write at specific indices, while the servers remain oblivious to which index was assigned to which writer. In particular, the setup ensures that none of the Remise servers learns (i) which users (from the superset of users including cover traffic and actual writers) are authorized to write messages, (ii) which indices are assigned to which writers, or (iii) the value of the write authorization token associated with an assigned index, which is issued to the authorized writer.

RemiseBB In RemiseBB, writers post messages on a shared bulletin board held by the servers which are reconstructed and published after every epoch. A writer submits a write request by sending shares of the message, shares of the index, and shares of the write authorization token. The servers accept the request if the token corresponds to the index (of course, without learning either the token or the index). Authorized indices in RemiseBB allow for preventing write-collisions as writers are only allowed to write at their assigned index. It also capitalizes on the asymmetry of real-world broadcasting, similar to what Spectrum [22] does, where there are fewer writers than total users in a system. Thus, while keeping the server computation only influenced by the number of real writers, we can still derive anonymity from all users without reserving extra space.

RemiseCC In contrast, in RemiseCC, while writers post messages similar to RemiseBB, the two servers never reconstruct the message database. Instead, the writers grant reading rights to specific readers of their choice, and this information is conveyed to the RemiseCC servers obliviously. Similar to write authorization tokens, the servers in RemiseCC also hold shares of read authorization tokens and the writer has control over the values of the read authorization tokens for the index they own. Readers submit read requests by sending shares of the index and shares of the read authorization tokens to the servers in order to retrieve a message. Upon receiving a read request, the RemiseCC servers verify (obliviously) whether the reader has permission to read from that index. If the permission is not granted, the read request is dropped.

Notation Throughout, for any value, array, or database D, we denote its additive sharing between servers $\mathtt{S}_0$ and $\mathtt{S}_1$ by $\mathrm{D} = \mathrm{D}_0^{\mathrm{AS}} + \mathrm{D}_1^{\mathrm{AS}}$, where $\mathtt{S}_b$ holds $\mathrm{D}_b^{\mathrm{AS}}$ for $b \in \{0,1\}$. If D is an additively shared array or database, then for every index $\mathtt{i}$, $\mathrm{D}[\mathtt{i}] = \mathrm{D}_0^{\mathrm{AS}}[\mathtt{i}] + \mathrm{D}_1^{\mathrm{AS}}[\mathtt{i}]$. We use D to denote both the bulletin board in RemiseBB and the message board in RemiseCC. The size of D is N. Servers $\mathtt{S}_0$ and $\mathtt{S}_1$ hold additive shares of D, as well as PRF-encrypted versions of the write-authorization token database, denoted by WRTOK. For each index $\mathtt{i}$, the entry $\mathrm{WRTOK}[\mathtt{i}]$ encodes the write-authorization tokens required to write to the $\mathtt{i}^{\text{th}}$ location of D. The database WRTOK is read-only. In RemiseCC, the servers additionally hold additive shares of the read-authorization tokens database, denoted by RDTOK. Each entry $\mathrm{RDTOK}[\mathtt{i}]$ consists of a list of m tokens, where possession of any one token suffices to prove authorization to read the contents of $\mathrm{D}[\mathtt{i}]$. During the setup phase, servers $\mathtt{S}_0$ and $\mathtt{S}_1$ hold additive shares of the setup database SETUPDB, to which writers post their public keys. At the end of setup, each writer learns its assigned indices via a published database denoted by ACCESSDB.

Threat Model The Remise clients (writers, readers and cover traffic) are completely untrusted. They may deviate arbitrarily from the protocol, form collusion with other clients or with a malicious server. Authorization tokens are treated as bearer credentials. Any entity possessing a valid read or write token for an index may exercise the corresponding capability. For privacy, Remise requires that the servers do not collude with each other. For integrity, Remise requires both servers must follow the protocol. We assume a global network adversary that observes all communication between clients and servers. We assume the existence of pseudorandom generators and pseudorandom functions for any randomness and key generation used in the protocols. We assume the existence of secure channels between each client and each server, providing confidentiality, integrity and replay protection for communicated messages.

3.1 Remise APIs

Remise exposes a set of APIs that the users use to interact with the system in both RemiseBB and RemiseCC. We model the system using the ideal functionality $\mathcal{F}_{\mathsf{Remise}}$. $\mathcal{F}_{\mathsf{Remise}}$ starts with invoking the SetupPhase containing a registration window during which the users may register as writers. Once the registration window closes, the functionality processes all the requests and assigns indices to registered writers. Once the setup phase ends, $\mathcal{F}_{\mathsf{Remise}}$ allows for writing, reading, and updating authorization any polynomial number of times and in arbitrary order.

Corrupted clients interact with the functionality through the adversary.

More specifically, $\mathcal{F}_{Remise}$ exposes four interfaces to the users: (i) Setup.Reg, (ii) Write, (iii) UpdateAuth, and (iv) Read. The functionality maintains the following state: (i) $\mathsf{WriteAuth} : [N] \to \{0,1\}^\lambda \cup \{\perp\}$, mapping each logical index to its write-authorization token, (ii) $\mathsf{ReadAuth} : [N] \to 2^{\{0,1\}^\lambda}$, mapping each logical index to its set of valid read-authorization tokens, and (iii) $\mathrm{D} : [N] \to \mathcal{M}$, storing the database contents at each logical index. Initially, all entries are set to $\perp$ (or the empty set where appropriate). It also maintains a clock T_{clock}, boolean variables, namely, setup_done and window_open.

Corruption. A PPT adversary $\mathcal{A}$ statically corrupts at most one server $\mathtt{S}_b$ ($b \in \{0,1\}$) and an arbitrary strict subset of clients $W_C \subsetneq W$, where W is the set of registered writers. We assume the existence of cover traffic, and that this cover traffic is not corrupted, since corrupting such a participant is equivalent to them not providing cover.

Real view. The real-world view of $\mathcal{A}$, denoted $\mathrm{View}^{\mathrm{Real}}_{\mathcal{A}}$, contains: (i) the inputs and random coins of every corrupted party, (ii) every protocol message sent or received by a corrupted party including its interaction with any ideal functionality invoked in the hybrid model, and (iii) the public transcript, i.e. every value published by the protocol.

Ideal world. In the ideal world, honest parties do not run the protocol; instead they forward their inputs to the trusted functionality $\mathcal{F}_{\mathsf{Remise}}$ through its interfaces. On each invocation $\mathcal{F}_{\mathsf{Remise}}$ updates its internal state, returns outputs to the parties over secure channels, and appends the corresponding entry to the leakage log $\mathtt{LeakLog}$.

The Simulator. The simulator $\mathcal{S}$ interacts with $\mathcal{F}_{\mathsf{Remise}}$ on behalf of the corrupted parties. The simulator does not know honest parties' inputs. It is given exactly: (i) the inputs and outputs of the corrupted parties (the corrupt server $\mathtt{S}_b$ and the clients in W_C), forwarded to and from $\mathcal{F}_{\mathsf{Remise}}$ on their behalf, and (ii) the leakage $\mathtt{LeakLog}$ produced by $\mathcal{F}_{\mathsf{Remise}}$ over the whole execution. From these alone $\mathcal{S}$ must produce a transcript $\mathrm{View}^{\mathsf{Ideal}}_{\mathcal{S}}$ that is indistinguishable from the adversary's real-world view. Security means $\mathrm{View}^{\mathrm{Real}}_{\mathcal{A}} \approx_c \mathrm{View}^{\mathsf{Ideal}}_{\mathcal{S}}$ for every PPT $\mathcal{A}$.

Setup Phase Remise (both RemiseBB and RemiseCC) provides a registration API that writers invoke during a one-time setup phase to obtain authorization to write to the database D. Each registered writer is assigned a database index together with a corresponding write authorization token, which will allow them to make authorized write requests to the database index. The setup phase hides from the servers both the set of registered writers and the indices assigned to them: while the servers may observe a superset of users submitting requests (including cover traffic), they cannot distinguish registered writers from non-writers. Additionally, the servers do not learn the value of any write authorization token.

$\mathcal{F}_{\mathsf{Remise}}$ begins by invoking the functionality SetupPhase (described in Phase 1). At the start of the setup phase, a registration window is open which is controlled by $\mathcal{F}_{\mathsf{Remise}}$'s internal clock T_{clock}. The end of the registration window, T_{end} is fixed in $\mathcal{F}_{\mathsf{Remise}}$. During the registration window, writers and cover-traffic participants may interact only with the $\mathsf{Setup.Reg}(\cdot,\cdot)$ interface (Interface 1). Registration requests submitted outside the active registration window are rejected and ignored by the functionality. The functionality accumulates all registration requests received during the window, and processes it accordingly when the window ends. The adversary learns the set of all participants but cannot distinguish registered writers ($\mathsf{mode} = \mathsf{register}$) from cover traffic ($\mathsf{mode} = \mathsf{cover}$). The adversary additionally learns the unordered set of assigned indices, but does not learn which participant received which index.

Definition 1 formalizes the anonymity achieved during the setup phase. The adversary learns the entire set of participants which includes at least one cover traffic provider and one honest writer but does not learn the set of registered writers.

Definition 1 (Setup anonymity)**.** *Let W be the set of registered writers in the setup phase and let $W_C \subsetneq W$ be the corrupted participants. The setup phase satisfies* setup anonymity *if, in the experiment of the Security Model, for every PPT adversary $\mathcal{A}$ corrupting one server and the participants in W_C, there exists a PPT simulator $\mathcal{S}$ such that* $\mathrm{View}^{\mathrm{Real}}_{\mathcal{A}} \approx_c \mathcal{S}(1^\lambda, W_C, \mathtt{LeakLog})$.

Write Operation in Remise In both RemiseBB and RemiseCC, registered writers are allowed to submit messages at their assigned locations. The system checks each submission and, if permitted, stores it in the message database. Users willing to provide cover traffic may also send write requests by sending *zero* messages. In the RemiseBB variant the bulletin board is published at the end of each epoch, whereas in RemiseCC, the message board is not published.

$\mathsf{Write}(\cdot,\cdot,\cdot)$ denotes the interface for writes; the formal definition appears in Interface 2. A client submits a write request by sending $(\mathtt{i}^*, \Delta\mathtt{M}, \omega)$, which are the target index, the value update, and the corresponding write authorization token. Here, $\Delta\mathtt{M} = \mathtt{M}' - \mathtt{M}$, where $\mathtt{M}'$ is the new value and $\mathtt{M}$ is the current value stored at $\mathrm{D}[\mathtt{i}^*]$. Since the authorized writer controls the value of $\mathrm{D}[\mathtt{i}^*]$, we assume that the writer knows the current value $\mathtt{M}$ at all times. $\mathsf{WriteAuth}[\mathtt{i}^*] = \omega$ means that ω is the correct write authorization token for writing at $\mathtt{i}^*$, while $\Delta\mathtt{M} = 0$ means

$\mathcal{F}_{\textbf{Remise}}$: **Phase 1** SetupPhase. Functionality of $\mathcal{F}_{Remise}$.

$P \leftarrow \emptyset$, $Q \leftarrow \emptyset$, $R \leftarrow \emptyset$
setup_done $\leftarrow$ false
window_open $\leftarrow$ true
while $T_{clock} \leq T_{end}$ **do**
// T_{clock} keeps ticking
// users use Setup.Reg to make requests
end while
window_open $\leftarrow$ false
for each $(w_j, \mathsf{mode}_j) \in P$ **do**
  if $\mathsf{mode}_j = \mathsf{register}$ **then**
    pick fresh unused index $\mathtt{i}^*_j \in [N]$
    $\omega_j \xleftarrow{\$} \{0,1\}^\lambda$
    $\mathsf{WriteAuth}[\mathtt{i}^*_j] \leftarrow \omega_j$
    $R \leftarrow R \cup \{\mathtt{i}^*_j\}$
    $Q \leftarrow Q \cup \{(w_j, \mathsf{mode}_j, \mathtt{i}^*_j, \omega_j)\}$
  end if
end for
// Leak unordered assigned-index set
$\mathtt{LeakLog} \leftarrow \mathtt{LeakLog} \,\|\, (\mathsf{assigned} : R)$
$\mathcal{A}$ learns LeakLog
for each $(w_j, \mathsf{mode_j}, \mathtt{i}^*_j, \omega_j) \in Q$ with $\mathsf{mode_j} = \mathsf{register}$ **do**
  w_j learns $(\mathtt{i}^*_j, \omega_j)$ over a secure channel
end for
setup_done $\leftarrow$ true

this is a zero message. In either case, ΔM is added to the $\mathtt{i}^{*\text{th}}$ index of D.

Updating Read Authorization in RemiseCC Unlike in RemiseBB, the message board in RemiseCC is not reconstructed and published. Instead, readers issue requests using read authorization tokens to read from the message board. The writers can decide who can read the messages they write. They do this by controlling the values of the read authorization tokens for the locations they are assigned to write at. Writers can then communicate values of the tokens to new readers, using some out-of-band method. Writers can also change the value of an existing read authorization token, thereby revoking access to all readers who were previously given that token. This step ensures that message access remains controlled and private.

$\mathsf{UpdAuth}(\cdot,\cdot,\cdot)$ denotes the interface for updating read authorization tokens for an index; the formal definition appears in Interface 3. The writer sends the element-wise difference of the new list of read authorization tokens with the current list. Since a writer authorized to write at index $\mathtt{i}^*$, controls the value of $\mathsf{ReadAuth}[\mathtt{i}^*]$, we assume that it knows the current value at all times.

$\mathcal{F}_{\textbf{Remise}}$: **Interface 1** $\mathsf{Setup.Reg}(w, \mathsf{mode})$: w is a user, bit $\mathsf{mode} \in \{\mathsf{register}, \mathsf{cover}\}$ tells if w wants to register or is providing cover traffic

if window_open = false **then**
  return $\perp$ to w // window closed
end if
$\mathtt{LeakLog} \leftarrow \mathtt{LeakLog} \,\|\, (\mathsf{Participant} : w)$
$P \leftarrow P \cup \{(w, \mathsf{mode})\}$
$\mathcal{A}$ learns LeakLog

$\mathcal{F}_{\textbf{Remise}}$: **Interface 2** $\mathsf{Write}(\mathtt{i}^*, \Delta M, \omega)$ from client c

if setup_done = false **then**
  return $\perp$
end if
// Either it is cover traffic or authorized write
if $(\Delta M = 0) \vee (\omega = \mathsf{WriteAuth}[\mathtt{i}^*])$ **then**
  out $\leftarrow$ acc
else
  out $\leftarrow$ rej
end if
// Leak public sender identity and decision
$\mathtt{LeakLog} \leftarrow \mathtt{LeakLog} \| (\mathsf{write} : c, \mathsf{out})$
$\mathcal{A}$ learns LeakLog
if out = acc **then**
  $\mathrm{D}[\mathtt{i}^*] \leftarrow \mathrm{D}[\mathtt{i}^*] + \Delta M$
end if
// Return final decision to the servers
Servers and c learn out

Read Operation in RemiseCC Readers who have been granted read access can request to retrieve messages using the read authorization tokens provided by the writer. The servers checks their authorization, and if valid, they provide the reader with the message content.

$\mathsf{Read}(\cdot,\cdot)$ denotes the interface for reads; the formal definition appears in Interface 4. A reader submits a read request by sending $(\mathtt{i}^*, \mathrm{t})$, which are the target index and a read-authorization token. If the token and the index submitted have the correct correspondence, i.e., $\mathrm{t} \in \mathsf{ReadAuth}[\mathtt{i}^*]$, then the read succeeds and the reader learns $\mathrm{D}[\mathtt{i}^*]$. The functionality leaks only the identity of the requesting reader and the accept/reject outcome through LeakLog.

Leakage profile. Recall that during setup phase, LeakLog contains entries of the form $(\mathsf{Participant} : w)$ and $(\mathsf{assigned} : R)$, where R denotes the unordered set of assigned indices. After the setup phase, arbitrary interleavings of Write, UpdAuth, and Read invocations append entries of the form $(\mathsf{write} : c, \mathsf{out})$, $(\mathsf{updauth} : c, \mathsf{out})$, and $(\mathsf{read} : c, \mathsf{out})$. The interface-specific leakage is captured entirely through the entries of LeakLog.

$\mathcal{F}_{\text{Remise}}$: **Interface 3** $\mathsf{UpdAuth}(\mathtt{i}^*, \Delta R, \omega)$ from client c

if setup_done = false **then**
  return $\perp$
end if
// $\Delta R \leftarrow R' - R$ is element-wise diff of new token list with current list
// Determine whether the update is accepted
if $\omega = \mathsf{WriteAuth}[\mathtt{i}^*]$ **then**
  $\mathsf{out} \leftarrow \mathsf{acc}$
else
  $\mathsf{out} \leftarrow \mathsf{rej}$
end if
// Leak public sender identity and decision
$\mathtt{LeakLog} \leftarrow \mathtt{LeakLog} \| (\mathsf{updauth} : c, \mathsf{out})$
if $\mathsf{out} = \mathsf{acc}$ **then**
  $\mathsf{ReadAuth}[\mathtt{i}^*] \leftarrow \mathsf{ReadAuth}[\mathtt{i}^*] + \Delta R$
end if
$\mathcal{A}$ learns LeakLog
// Return final decision to the servers
Servers and c learn out

$\mathcal{F}_{\text{Remise}}$: **Interface 4** $\mathsf{Read}(\mathtt{i}^*, t)$ from client c

if setup_done = false **then**
  return $\perp$
end if
// Determine whether the read is accepted
if $t \in \mathsf{ReadAuth}[\mathtt{i}^*]$ **then**
  $\mathsf{out} \leftarrow \mathsf{acc}$ // t is a valid token at $\mathtt{i}^*$
else
  $\mathsf{out} \leftarrow \mathsf{rej}$
end if
// Leak public reader identity and decision
$\mathtt{LeakLog} \leftarrow \mathtt{LeakLog} \| (\mathsf{read} : c, \mathsf{out})$
$\mathcal{A}$ learns LeakLog
if $\mathsf{out} = \mathsf{acc}$ **then**
  c learns $\mathrm{D}[\mathtt{i}^*]$
else
  c learns $\perp$
end if
// Return final decision to the servers
Servers and c learn out

4 Remise: The Details

Protocols 5 to 8 realize the interfaces and the authorization states of the ideal functionality $\mathcal{F}_{\mathsf{Remise}}$ described in Section 3.1. WriteAuth is realized by WrTok, which is a DOROM held by $\mathtt{S}_0$ and $\mathtt{S}_1$. ReadAuth is realized by RdTok, a DORAM database held by $\mathtt{S}_0$ and $\mathtt{S}_1$, storing a list of m tokens per index. In the remainder of this section, we describe these protocols in detail and present their corresponding security theorems.

Setup Phase in Remise The setup phase of Remise (both RemiseBB and RemiseCC) consists of three stages. It begins with a *writer registration stage*, which follows a bootstrapping mechanism similar to that of Spectrum [22] and relies on an anonymous broadcast system such as Riposte [7]. The servers $\mathtt{S}_0$ and $\mathtt{S}_1$ hold additive shares of an anonymous bulletin board SetupDB. Writers anonymously submit public keys to random locations in SetupDB using a Riposte-style anonymous broadcast, modeled by the ideal functionality $\mathcal{F}_{AB}$. Anonymous broadcast hides the link between the public key pk and the broadcasting writer w. Writes to SetupDB are accepted only during a fixed registration window.

Remise inherits a non-zero probability of failed registrations due to write collisions in Riposte. However, since the registration occurs only once during setup, this allows for us to make the failure probability negligible by over-provisioning the database relative to the maximum number of writer registrations allowed in Remise. We can also use collision recovery methods, as suggested in Riposte [7]. For example, a table of size $2.7N$ achieves roughly 95% successful registrations while tolerating two-way collisions, where N is the maximum number of registrations.

Once the writer registration phase ends, the servers reconstruct SetupDB and recover the set of public keys written at random locations. For each such pk, the servers assign a unique database index $\mathtt{i}^*$ using an agreed-upon ordering between $\mathtt{S}_0$ and $\mathtt{S}_1$. The servers hold the additive shares of the write authorization token database WrTok. To generate a write authorization token for $\mathtt{i}^*$ without revealing it to either servers, $\mathtt{S}_0$ and $\mathtt{S}_1$ independently sample random strings ℓ_0 and ℓ_1 respectively, without learning each other's shares. $\ell_0 + \ell_1 = \ell$ becomes the write authorization token for $\mathtt{i}^*$. They locally update their respective shares of WrTok by setting $\mathrm{WrTok}_0[\mathtt{i}^*] \leftarrow \ell_0$ and $\mathrm{WrTok}_1[\mathtt{i}^*] \leftarrow \ell_1$. After assigning an index and token for all submitted requests, the servers convert their additive shares of WrTok into doubly encrypted read-only database denoted by $\mathrm{WrTok}^{\mathrm{RO}}$.

Finally, for each writer with public key pk, each server encrypts its share of the authorization token together with the assigned index $\mathtt{i}^*$ under pk. These ciphertexts are written to a separate database AccessDB at the same position from which pk was read in SetupDB. The published database AccessDB concludes the setup phase, enabling each writer to decrypt and learn its assigned index and corresponding write authorization token. Figure 2 illustrates the setup phase in Remise.

Protocol 5 describes the setup phase in detail. Although the protocol shows w registering for a single index, any writer can reserve multiple indices by making more than one request. The servers may enforce a limit

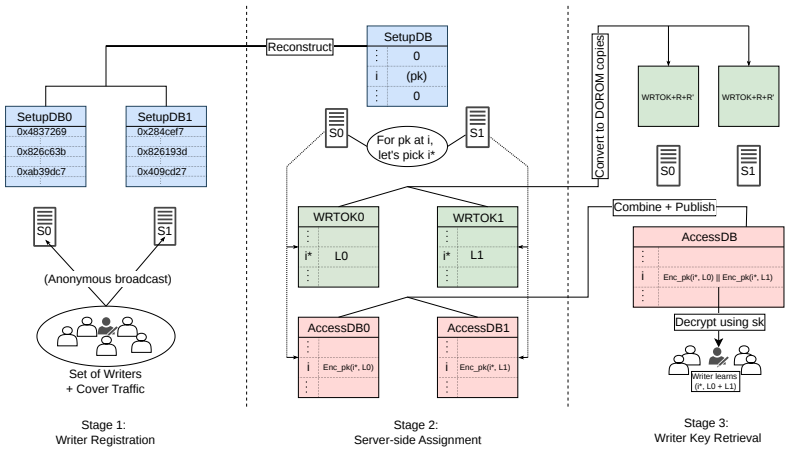


Figure 2: Remise setup phase: In Stage 1: writers post their public keys using an anonymous broadcast system like Riposte. In Stage 2: servers get the value of public key and assign an index $\mathtt{i}^*$. They also sample their respective shares of the write authorization token ($L0$ and $L1$), so that no single server knows the actual value of the token. In Stage 3: the servers combine their respective encrypted shares and publish it. They also convert WRTOK into DOROM copies.

on the number of requests made by each user during the setup phase, to prevent a user from exhausting server resources. In line 5, the servers jointly assign each posted public key to a unique database index by evaluating a randomized function ASSIGN, agreed upon by servers $\mathtt{S}_0$ and $\mathtt{S}_1$. This function picks a uniformly random index and discards a sampling if the index is already assigned.

Note that during Stage 1, every writer interacts with the anonymous-broadcast functionality $\mathcal{F}_{\mathsf{AB}}$: a *registering* writer broadcasts a public key, while a *cover-traffic* participant broadcasts a zero message over the same channel. By the anonymity of $\mathcal{F}_{\mathsf{AB}}$, a corrupt server cannot distinguish a public-key broadcast from a zero broadcast, and hence cannot tell a registrant from a cover participant; this indistinguishability is inherited from $\mathcal{F}_{\mathsf{AB}}$ and is not a property of $\mathcal{F}_{\mathsf{Remise}}$ itself. What $\mathcal{F}_{\mathsf{Remise}}$ additionally guarantees is that the *set* of assigned indices is revealed without any link to the writers who received them (Definition 1): even an adversary corrupting one server and the writers in W_C learns the assigned-index set and its own assignments, but no correspondence for any honest writer(s).

Theorem 1 (Setup anonymity)**.** *Assume* PRF *is a secure pseudorandom function, the encryption scheme is IND-CPA secure, and $\mathcal{F}_{AB}$ is the ideal functionality for anonymous broadcast. Then Protocol 5 securely realizes* SetupPhase *(Phase 1) of $\mathcal{F}_{Remise}$ with respect to Definition 1, against any PPT adversary corrupting at most one server and any strict subset $W_C \subsetneq W$ of writers.*

Proof sketch. WLOG assume $\mathtt{S}_0$ is corrupt. Given the leakage from SetupPhase as LeakLog and the corrupted writer set W_C, the simulator $\mathcal{S}$ constructs the adversary's view as follows.

(i) *Simulating identities and assignment.* $\mathcal{S}$ processes LeakLog sequentially. For each leakage entry of the form $(\mathsf{Participant} : w)$, $\mathcal{S}$ records that participant w participated in the registration. When $\mathcal{S}$ encounters an entry $(\mathsf{assigned} : R)$, it interprets R as the unordered set of assigned indices. $\mathcal{S}$ now simulates a mapping between public keys and assigned indices for the setup phase as follows: for corrupted writers $w \in W_C$, $\mathcal{S}$ uses the adversary-provided public keys and assigned indices. For the remaining assigned indices (corresponding to honest registrants), $\mathcal{S}$ samples independent uniformly random public keys, and defines $\{\mathtt{pk}_w \mapsto \mathtt{i^*}_w\}_{\mathtt{i^*}_w \in R}$, (ii) *Simulating* SETUPDB. $\mathcal{S}$ populates SETUPDB with public keys for all $w \in W_C$. The remaining public keys simulated for the honest writers are given to the simulator for $\mathcal{F}_{\mathsf{AB}}$, which generates a transcript where the keys appear at uniformly random locations (other than the ones previously populated) in the corrupt server's view of SETUPDB, leaking only the information allowed by $\mathcal{F}_{\mathsf{AB}}$. By anonymity of $\mathcal{F}_{\mathsf{AB}}$, the simulated SETUPDB is computationally indistinguishable from the real execution, (iii) *Simulating* ACCESSDB *and local state.* Using the mapping $\{\mathtt{pk}_w \mapsto \mathtt{i^*}_w\}_{\mathtt{i^*}_w \in R}$, $\mathcal{S}$ constructs the corresponding entries of ACCESSDB: For all assigned indices, $\mathcal{S}$ knows ℓ_0 corresponding to $\mathtt{i^*}_w$ (i.e., the value in $\mathrm{WRTOK}_0[\mathtt{i^*}_w]$). For indices belonging to honest writers, it samples a uniformly random string to simulate ℓ_1 (the value in $\mathrm{WRTOK}_1[\mathtt{i^*}_w]$), while for writers in W_C, it uses adversary known values The simulator writes the encryption of $(\mathtt{i^*} \| \ell_0)$ and $(\mathtt{i^*} \| \ell_1)$, using $\mathtt{pk}_w$., and (iv) *Simu-*

Protocol 5 SETUP PHASE IN REMISE: A writer w requests write authorization to an index in D. Servers $\mathtt{S}_0$ and $\mathtt{S}_1$ hold secret shares of a setup bulletin board, SETUPDB. A write authorization token database, WRTOK, is also secret shared between the two servers. WRTOK is empty at the start of the protocol.

STAGE 1: Writer Registration

1: w generates key pair $(\mathtt{pk}, \mathtt{sk})$
2: w writes $\mathtt{pk}$ to a random location $\mathtt{i}$ in SETUPDB, i.e. SETUPDB$[\,\mathtt{i}\,] \leftarrow \mathtt{pk}$, using a Riposte-styled write

STAGE 2: Server-Side Assignment

// After the writer-registration window ends
3: $\mathtt{S}_0$ and $\mathtt{S}_1$ reconstruct SETUPDB
4: **for all** $\mathtt{i} \in [|\text{SETUPDB}|]$ such that SETUPDB$[\,\mathtt{i}\,]$ is valid public key $\mathtt{pk}$ **do**
// Servers agree on assignment of fresh index
5: $\mathtt{S}_0$ and $\mathtt{S}_1$ compute $\mathtt{i}^* \leftarrow \text{ASSIGN}()$
6: **for** $b \in \{0,1\}$ **do**
7: $\mathtt{S}_b$ samples a random string $\ell_b \xleftarrow{\$} \{0,1\}^\lambda$
8: $\mathtt{S}_b$ stores $\text{WRTOK}_b[\mathtt{i}^*] \leftarrow \ell_b$
9: $\mathtt{S}_b$ computes $c_b \leftarrow \text{ENC}_{\mathtt{pk}}(\mathtt{i}^* \,||\, \ell_b)$
10: **end for**
11: $\mathtt{S}_0$ and $\mathtt{S}_1$ write $(c_0 \,||\, c_1)$ to ACCESSDB$[\,\mathtt{i}\,]$
12: **end for**
13: $\mathtt{S}_0$ and $\mathtt{S}_1$ publish ACCESSDB
// servers convert WRTOK shares into encrypted copies
14: For each $b \in \{0,1\}$, $\mathtt{S}_b$ samples key $\mathtt{k}_b$
15: $\mathtt{S}_b$, constructs vector R_b, where $R_b[\,\mathtt{i}'] = \mathtt{PRF}(\mathtt{k}_b, \mathtt{i}')$ for all index $\mathtt{i}' \in [N]$
16: $\mathtt{S}_b$ sends $(\text{WRTOK}_b + R_b)$ to $\mathtt{S}_{1-b}$
// $\mathtt{S}_0$, $\mathtt{S}_1$ hold $(\text{WRTOK}_0 + R_0), (\text{WRTOK}_1 + R_1)$
17: Servers calculate and store $\text{WRTOK}^{\text{RO}} \leftarrow \text{WRTOK}_0 + R_0 + \text{WRTOK}_1 + R_1$

STAGE 3: Writer Key Retrieval

18: w retrieves ACCESSDB$[\,\mathtt{i}\,]$
19: w decrypts using $\mathtt{sk}$ to obtain $(\mathtt{i}^*, \ell_0, \ell_1)$
20: w reconstructs $\ell \leftarrow \ell_0 + \ell_1$ // ℓ is the write authorization token for $\mathtt{i}^*$

lating WRTOK^{RO}. Finally, it simulates the PRF-masked read-only memory, by simulating the masked share of $\mathtt{S}_1$ $(\text{WRTOK}_1 + R_1)$ by replacing PRF values (R_1) with uniformly random values. Finally, $\mathcal{S}$ outputs the simulated SETUPDB, ACCESSDB, WRTOK^{RO}, and the local state of $\mathtt{S}_0$.

The simulated view is computationally indistinguishable from the real execution by: (i) anonymity of $\mathcal{F}_{\mathsf{AB}}$, (ii) pseudorandomness of $\mathtt{PRF}$, and (iii) IND-CPA security of the encryption scheme. All remaining components are perfectly distributed. □

Writing in Remise A writer submits a *write* request to servers $\mathtt{S}_0$ and $\mathtt{S}_1$ by sending $(\mathtt{i}^{*\text{AS}}, \Delta\mathtt{M}^{\text{AS}}, \omega^{\text{AS}})$, which are additive shares of the target index, the value update, and the corresponding write authorization token. $\text{D}[\mathtt{i}^*]$ is updated with value $\Delta\mathtt{M}$ if either ω is the valid write authorization token for $\mathtt{i}^*$, or if $\Delta\mathtt{M} = 0$, corresponding to cover traffic.

Protocol 6 describes the process of writing in Remise. w has full control over the value $\text{D}[\,\mathtt{i}^*\,]$ and thus we assume it knows the value $\mathtt{M}$ before making write request. Before any write is made $\text{D}[\,\mathtt{i}\,] = 0$ for all index $\mathtt{i}$. Line 3 checks if the payload value ($\Delta\mathtt{M}$), is zero. If $\Delta\mathtt{M}$ is zero, $z^{\text{AS}} \leftarrow 1^{\text{AS}}$, otherwise $z^{\text{AS}} \leftarrow 0^{\text{AS}}$. Line 4 does a DOROM read into the token database to get shares of the actual token of write authorization. Line 6 computes an MPC OR of two check values, z and auth, to ensure either a zero message is provided or the authorization passed. In RemiseBB, Line 9 is a write-only memory access, while in RemiseCC, it is a DORAM write.

Providing Read Authorization Granting read access to an index $\mathtt{i}^*$ in RemiseCC consists of two steps: (i) distributing a read-authorization token to the reader, and (ii) obliviously updating the token database so that possession of the token grants read access to $\mathtt{i}^*$. A writer may distribute the token either through an initial in-person exchange or via a dialing protocol such as Express [13].[2] After distributing the token, the writer must inform the servers that possession of the token authorizes reading from an index (say $\mathtt{i}^*$) of the message board. Servers $\mathtt{S}_0$ and $\mathtt{S}_1$ hold additive shares of a read authorization token database RDTOK. Each entry $\text{RDTOK}[\mathtt{i}]$ consists of a list of m tokens, where possession of any one token suffices to read $\text{D}[\mathtt{i}]$. Before updating RDTOK, the servers verify that the requester is authorized to write to index $\mathtt{i}^*$ by checking the supplied write-authorization proof, exactly as in the write operation (Protocol 6). If the check succeeds, $\mathtt{S}_0$ and $\mathtt{S}_1$ update their shares of RDTOK via a DORAM write. The writer can revoke read authorization

[2] While one may ask why messages themselves are not shared this way, note that the expensive setup is required only once.

Protocol 6 WRITE IN REMISE: Writer w wants to write $\mathtt{M}'$ at $\mathtt{i}^{*\text{th}}$ index of the database D. w knows the current value of $\mathrm{D}[\,\mathtt{i}^*\,]$ as $\mathtt{M}$ and holds ω as write authorization proof. $\mathtt{S}_0$ and $\mathtt{S}_1$ hold shares of D and WRTOK. At the end of the protocol, if the write authorization passes, $\mathtt{S}_0$ and $\mathtt{S}_1$ will end up with new shares of D, such that $\mathrm{D}[\,\mathtt{i}^*\,] \leftarrow \mathtt{M}'$

1: w calculates $\Delta\mathtt{M} \leftarrow \mathtt{M}' - \mathtt{M}$ // new val. - curr. val.
2: For each $b \in \{0,1\}$, w sends $(\mathtt{i}^{*\mathrm{AS}}_b, \Delta\mathtt{M}^{\mathrm{AS}}_b, \omega^{\mathrm{AS}}_b)$ to $\mathtt{S}_b$.
// Servers obliviously check for zero message
3: $z^{\mathrm{AS}} \leftarrow \mathrm{EQZ}(\Delta\mathtt{M}^{\mathrm{AS}})$
// Servers do DOROM read on the write token database
4: $\mathcal{F}^{\mathrm{AS}} \leftarrow \mathrm{WRTOK}^{\mathrm{RO}}[\mathtt{i}^{*\mathrm{AS}}]$
5: $\mathrm{auth}^{\mathrm{AS}} \leftarrow \mathrm{EQZ}(\mathcal{F}^{\mathrm{AS}} - \omega^{\mathrm{AS}})$
// MPC OR operation
6: $\mathrm{audit}^{\mathrm{AS}} \leftarrow \mathrm{auth}^{\mathrm{AS}} \vee z^{\mathrm{AS}}$
// Servers reconstruct audit
7: $\mathrm{audit} \leftarrow \mathrm{REC}(\mathrm{audit}^{\mathrm{AS}})$
8: **if** audit = 1 **then**
9: $\quad \mathrm{D}^{\mathrm{AS}}[\mathtt{i}^{*\mathrm{AS}}] \leftarrow \mathrm{D}^{\mathrm{AS}}[\mathtt{i}^{*\mathrm{AS}}] + \Delta\mathtt{M}^{\mathrm{AS}}$ // $\mathrm{D}[\,\mathtt{i}^*\,] = \mathtt{M}$
10: **else**
11: $\quad$ **return** // Reject the request and perform no write
12: **end if**

for readers holding a token, by removing it from the list $\mathrm{RDTOK}[\mathtt{i}^*]$ during the update. To support cover traffic[3], RemiseCC designates a public-update index $\mathtt{i}_{\mathrm{upd}}$ whose write-authorization token is public. Any user willing to provide cover traffic can make updates to $\mathrm{RDTOK}[\mathtt{i}_{\mathrm{upd}}]$ using the same protocol for update (Protocol 7). The servers do not assign $\mathtt{i}_{\mathrm{upd}}$ in the setup phase.

Protocol 7 describes the protocol that allows a writer to update the values of read authorization tokens for an index. The RemiseCC servers have to first ascertain that the writer updating the tokens for an index, is indeed the author of the messages at that index (say $\mathtt{i}^*$). Lines 4 to 6 authenticate the writer. Once the writer is authenticated, in Line 8 RDTOK is updated via a DORAM update operation, such that the list of tokens at index $\mathtt{i}^*$ holds new token values that the writer wants.

Reading in RemiseCC Recall that in RemiseCC, the servers do not reconstruct the message board. Instead, readers read from D, held as additive shares by $\mathtt{S}_0$ and $\mathtt{S}_1$. A reader submits a *read* request by sending $(\mathtt{i}^{*\mathrm{AS}}, \mathtt{t}^{\mathrm{AS}})$, which are shares of the index they wish to read from and the shares of the read authorization token they received from the writer. Protocol 8 describes the protocol to read

[3]Without cover traffic, valid updates to RDTOK reveals the set of registered writers in the system.

Protocol 7 UPDATE READ AUTHORIZATION IN REMISECC: Writer w has write authorization to index $\mathtt{i}^*$ and knows the current list of m tokens at $\mathrm{RDTOK}[\,\mathtt{i}^*\,]$ denoted by $R_{\mathtt{i}^*}$. $\mathtt{S}_0$ and $\mathtt{S}_1$ hold shares of a token database RDTOK. w wants to write new list $R'_{\mathtt{i}^*}$ at $\mathrm{RDTOK}[\,\mathtt{i}^*\,]$.

// w prepares new tokens for $\mathtt{i}^*$.
1: w constructs $R'_{\mathtt{i}^*} = \{\mathtt{t}_1, \ldots, \mathtt{t}_m\}$.
// w computes the element-wise difference
2: $\Delta R_{\mathtt{i}^*} \leftarrow R'_{\mathtt{i}^*} - R_{\mathtt{i}^*}$.
3: For each $b \in \{0,1\}$, w sends $(\mathtt{i}^{*\mathrm{AS}}, \Delta R_{\mathtt{i}^*}{}^{\mathrm{AS}}, \omega^{\mathrm{AS}})$
// Servers check writer's authorization.
4: $\mathcal{F}^{\mathrm{AS}} \leftarrow \mathrm{WRTOK}^{\mathrm{AS}}[\mathtt{i}^{*\mathrm{AS}}]$
5: $\mathrm{check}^{\mathrm{AS}} \leftarrow \mathrm{EQZ}(\mathcal{F}^{\mathrm{AS}} - \omega^{\mathrm{AS}})$
6: $\mathrm{check} \leftarrow \mathrm{REC}(\mathrm{check}^{\mathrm{AS}})$
7: **if** check = 1 **then**
// Authorized: update token list at $\mathtt{i}^*$.
8: $\quad \mathrm{RDTOK}^{\mathrm{AS}}[\mathtt{i}^*] \leftarrow \mathrm{RDTOK}^{\mathrm{AS}}[\mathtt{i}^*] + \Delta R_{\mathtt{i}^*}{}^{\mathrm{AS}}$
9: **else**
10: $\quad$ **return** // Reject the request
11: **end if**
// Cover traffic: any user may run the same protocol with $\mathtt{i}^* = \mathtt{i}_{\mathrm{upd}}$, using the public write-token for $\mathtt{i}_{\mathrm{upd}}$.

in RemiseCC. We first remark that the notion of reading in RemiseBB does not exist since that is a broadcasting system, where the messages are published. Lines 2 to 11 authenticate the reader. To authenticate, the servers obliviously check that the reader knows at least one of the read tokens associated with the index they are trying to access. Readers providing cover traffic simply send read requests to a special index $\mathtt{i}_{\mathrm{read}}$. Similar to $\mathtt{i}_{\mathrm{upd}}$, this index is not assigned to a writer during the setup phase. The read authorization tokens $\mathtt{i}_{\mathrm{read}}$ ($\mathrm{RDTOK}[\mathtt{i}_{\mathrm{read}}]$), are made public and are used to make read requests to it. The write authorization token for $\mathtt{i}_{\mathrm{read}}$ ($\mathrm{WRTOK}[\mathtt{i}_{\mathrm{read}}]$), however, must be hidden by the servers. This is to prevent any malicious user from updating $\mathrm{RDTOK}[\mathtt{i}_{\mathrm{read}}]$ using Protocol 7.

Theorem 2 (Write/UpdAuth/Read privacy)**.** *In the* $(\mathcal{F}_{\mathrm{DOROM}}, \mathcal{F}_{\mathrm{DOWOM}}, \mathcal{F}_{\mathrm{DORAM}}, \mathcal{F}_{\mathrm{EQZ}}, \mathcal{F}_{\mathrm{MULT}})$*-hybrid model, Protocols 6, 7, and 8 securely realize the* Write*,* UpdAuth*, and* Read *interfaces, respectively, of* $\mathcal{F}_{\mathit{Remise}}$ *against any semi-honest adversary corrupting one server and any subset of clients.*

Proof sketch. WLOG assume $\mathtt{S}_0$ is corrupt. The simulator $\mathcal{S}$ processes LeakLog sequentially. For each leakage entry of the form $(\mathsf{write} : c, \mathsf{out})$, $(\mathsf{updauth} : c, \mathsf{out})$, or $(\mathsf{read} : c, \mathsf{out})$, $\mathcal{S}$ invokes the corresponding simulation procedure described below to simulate the adversary's view using the underlying hybrid simulators.

Write. Let $(\mathtt{i}^*, \Delta M, \omega)$ be the honest writer's input. The simulator samples additive shares $\mathtt{i}^{*\mathrm{AS}}_0$, $\Delta\mathtt{M}^{\mathrm{AS}}_0$, and

Protocol 8 READ IN REMISECC: A reader r wants to read content $\mathrm{D}[\,\mathtt{i}^*\,]$ using read authorization token t. Servers hold shares of D and RDTOK. At the end of the protocol, r reads $\mathrm{D}[\mathtt{i}^*]$ if $\mathsf{t} \in \mathrm{RDTOK}[\mathtt{i}^*]$.

// Reader sends shares of index and its read authorization token to the servers
// Readers providing cover traffic would send same request for the reserved index $\mathtt{i}_{\text{read}}$
1: For each $b \in \{0,1\}$, r sends $(\mathtt{i}^{*\mathrm{AS}}_b, \mathsf{t}^{\mathrm{AS}}_b)$ to $\mathtt{S}_b$
// Servers do a DORAM read to get shares of the list of tokens for $\mathtt{i}^*$
2: $T^{\mathrm{AS}} \leftarrow \mathrm{RDTOK}^{\mathrm{AS}}[\mathtt{i}^{*\mathrm{AS}}]$
3: Parse $T^{\mathrm{AS}} = \{\mathsf{t}_1{}^{\mathrm{AS}}, \ldots, \mathsf{t}_m{}^{\mathrm{AS}}\}$
// Servers calculate additive shares of the check value for each of the token in the list
4: **for all** $\mathsf{t}_i \in T$ **do**
5: $\quad \mathrm{check}_i{}^{\mathrm{AS}} \leftarrow \mathsf{t}_i{}^{\mathrm{AS}} - \mathsf{t}^{\mathrm{AS}}$
6: **end for**
// Servers check if any token matches
7: **for** $i = 1$ to m **do**
8: $\quad z_i{}^{\mathrm{AS}} \leftarrow \mathrm{EQZ}(\mathrm{check}_i{}^{\mathrm{AS}})$
9: **end for**
10: $\mathrm{auth}^{\mathrm{AS}} \leftarrow \bigvee_{i=1}^{m} z_i{}^{\mathrm{AS}}$
11: $\mathrm{auth} \leftarrow \mathrm{REC}(\mathrm{auth}^{\mathrm{AS}})$ // servers reconstruct
12: **if** auth = 1 **then**
// Allow read operation
13: $\quad \mathrm{read}^{\mathrm{AS}} \leftarrow \mathrm{D}^{\mathrm{AS}}[\mathtt{i}^{*\mathrm{AS}}]$
14: $\quad \mathtt{S}_0$ and $\mathtt{S}_1$ send $\mathrm{read}_0^{\mathrm{AS}}$ and $\mathrm{read}_1^{\mathrm{AS}}$ to the reader.
// Reconstruction of the shares
15: $\quad$ The reader computes: $\mathrm{read} \leftarrow \mathrm{read}_0^{\mathrm{AS}} + \mathrm{read}_1^{\mathrm{AS}}$
16: **else**
// Drop the read request
17: $\quad$ **return**
18: **end if**

ω_0^{AS} uniformly and delivers them to $\mathtt{S}_0$. It then: (i) invokes the $\mathcal{F}_{\mathrm{EQZ}}$ simulator to simulate the zero-check on $\Delta\mathtt{M}^{\mathrm{AS}}$, (ii) invokes the $\mathcal{F}_{\mathrm{DOROM}}$ simulator on $(1^\lambda, \mathtt{i}^{*\mathrm{AS}}_0)$ to simulate retrieval of the proof share, (iii) invokes the $\mathcal{F}_{\mathrm{EQZ}}$ simulator again for the proof equality test, (iv) invokes the $\mathcal{F}_{\mathrm{MULT}}$ simulator for the OR-combination step;, and (v) if $\mathsf{out} = \mathsf{acc}$, invokes the write simulator for either $\mathcal{F}_{\mathrm{DOWOM}}$ (RemiseBB) or $\mathcal{F}_{\mathrm{DORAM}}$ (RemiseCC) on $(1^\lambda, \mathtt{i}^{*\mathrm{AS}}_0, \Delta\mathtt{M}^{\mathrm{AS}}_0)$. If $\mathsf{out} = \mathsf{rej}$, the simulator halts.

UpdAuth. Let $(\mathtt{i}^*, \Delta R_{\mathtt{i}^*}, \omega)$ be the honest writer's input. The simulator samples $\mathtt{i}^{*\mathrm{AS}}_0$, ω_0^{AS}, and $\Delta R_{\mathtt{i}^*0}{}^{\mathrm{AS}}$ uniformly and delivers them to $\mathtt{S}_0$. It then: (i) invokes the $\mathcal{F}_{\mathrm{DOROM}}$ simulator on $(1^\lambda, \mathtt{i}^{*\mathrm{AS}}_0)$ to simulate retrieval of the proof share, (ii) invokes the $\mathcal{F}_{\mathrm{EQZ}}$ simulator to simulate the equality test, and (iii) if $\mathsf{out} = \mathsf{acc}$, invokes the $\mathcal{F}_{\mathrm{DORAM}}$ simulator on $(1^\lambda, \mathtt{i}^{*\mathrm{AS}}_0, \Delta R_{\mathtt{i}^*0}{}^{\mathrm{AS}})$ to simulate the token-database update. If $\mathsf{out} = \mathsf{rej}$, the simulator halts.

Read. Let $(\mathtt{i}^*, \mathsf{t})$ be the honest reader's input. The simulator samples $\mathtt{i}^{*\mathrm{AS}}_0$ and $\mathsf{t}^{\mathrm{AS}}_0$ uniformly and delivers them to $\mathtt{S}_0$. It then: (i) invokes the $\mathcal{F}_{\mathrm{DORAM}}$ simulator on $(\mathtt{i}^{*\mathrm{AS}}_0, \mathsf{t}^{\mathrm{AS}}_0)$ to simulate retrieval of token shares, (ii) invokes the $\mathcal{F}_{\mathrm{EQZ}}$ simulator for the m equality checks, (iii) invokes the $\mathcal{F}_{\mathrm{MULT}}$ simulator for the m-way-OR-combination step, and (iv) if $\mathsf{out} = \mathsf{acc}$, invokes the $\mathcal{F}_{\mathrm{DORAM}}$ simulator on $(1^\lambda, \mathtt{i}^{*\mathrm{AS}}_0)$ to simulate the DORAM read. If $\mathsf{out} = \mathsf{rej}$, the simulator halts.

In all three cases, the simulated shares are uniformly distributed and computationally indistinguishable from the real protocol execution by the security of the underlying hybrid functionalities. Hence, the simulated transcript is indistinguishable from the corrupt server's real view. □

We end this section by remarking that Dynamic broadcast encryption schemes such as that of Delerablée et al. [10] also achieve authorized reading by restricting decryption to authorized readers, but typically reveal authorization or revocation structure through ciphertext or key updates. In contrast, RemiseCC stores authorization state separately in the oblivious token database RDTOK. Revocation requires only an oblivious DORAM update to RDTOK, without re-encrypting stored messages or revealing which authorization state changed.

4.1 RemiseBB (Sabre): Authorization w/o DORAMs

If we relax our threat model to disallow collusion between a RemiseBB server and a user, we can achieve authorization without relying on DORAMs by borrowing Sabre's authorization check [31]. Let $F : \{0,1\}^\lambda \times I \to \{0,1\}^\lambda$ be a PRF. The authorization token for an index i is defined as $\tau_i = F_{\mathtt{k}}(i)$, where the PRF key $\mathtt{k}$ is known to the two servers. A client authorized to access index $\mathtt{i}^*$ is given the token $\tau_{\mathtt{i}^*} = F_{\mathtt{k}}(\mathtt{i}^*)$ during setup. To make an access, the client submits additive shares of the target index $\mathtt{i}^{*\mathrm{AS}}_0, \mathtt{i}^{*\mathrm{AS}}_1$, along with additive shares of the token $\tau_0^{\mathrm{AS}}, \tau_1^{\mathrm{AS}}$. Using their shares of $\mathtt{i}^*$, the servers jointly evaluate the PRF under MPC to compute $\tau^\star = F_{\mathtt{k}}(\mathtt{i}^*)$, and then perform an MPC equality check to verify that $\tau^\star = \tau$. If the check succeeds, the access is authorized. The authorization check has constant computational complexity and communication complexity. As in Sabre, collusion between a server and a client is out of scope: since the servers know the PRF key, they can compute valid authorization tokens for any index, and therefore such collusion trivially enables unauthorized access. Unlike Sabre, RemiseBB (Sabre) does not need to perform an audit to check the validity of the DPFs. We remark that combining RemiseCC with Sabre does not yield an efficient construction that also preserves writer-reader unlinkability.

4.2 Limitations

Concurrent Reads and Writes. RemiseCC does not guarantee linearizable consistency under concurrent reads and writes to the same location. If a read and write overlap, the two servers may observe different operation orders, causing the reader to reconstruct one old share and one updated share, yielding an inconsistent value. This affects only read consistency; it does not compromise write correctness, authorization, or privacy. Once both servers process the write, the database converges to the correct state. This limitation is common in asynchronous secret-shared storage systems without coordination, including Express and Sabre. Systems such as Riposte and Spectrum avoid this issue by restricting reads to completed epochs. Locking or versioning could provide stronger consistency, but would increase latency and potentially leak additional access-pattern information. We leave stronger consistency guarantees to future work.

Out-of-Band Token Distribution. When granting read access, a writer in RemiseCC must distribute a fresh authorization token through an out-of-band channel. This assumption is also present in systems such as Sabre, Express, and Spectrum, where readers must initially obtain shared secrets or authorization material externally. Designing scalable and privacy-preserving authorization distribution mechanisms without trusted setup channels remains an open problem.

Malicious Servers. Unlike in Spectrum, Remise does not provide any Blame-Game protocol. Therefore, a malicious server could potentially drop write request from an honest client. We assume semi-honest behavior from Remise servers.

5 Implementation and Evaluation

Table 2 describes the asymptotic costs in realizing Remise. DPF evaluation in MPC accounts for the preprocessing cost. Notice that the costs in RemiseCC have factor of m, the number of read tokens per database index.

We implemented a proof-of-concept prototype in C++. All experiments were conducted on a server equipped with two *Intel® Xeon® Gold 6430* processors, 251 GiB of RAM, and running a 64-bit *Linux* operating system. Network latency and bandwidth were emulated using `tc netem`, and all experiments were run on an otherwise idle system. Spectrum (PACL) and Express (PACL) mean Spectrum and Express integrated with PACL access control mechanism, respectively. We compare RemiseBB with Spectrum (PACL), and RemiseBB (Sabre) with Express (PACL), since each pair operates under the same threat model. In particular, in both RemiseBB and Spectrum (PACL), server-client collusion does not allow the server to unilaterally issue a write token to an arbitrary writer. While in RemiseBB (Sabre) and Express (PACL), server-client collusion can enable a single server to issue write tokens to arbitrary writers.

In the following, we compare Remise with the various anonymous communication systems in PACL's codebase[4], namely Spectrum and Express.

Bandwidth Comparisons Table 3 compares the bandwidth consumption for write operations of Remise and PACL systems. First observe that the client request size for write operations in Remise (RemiseBB, RemiseCC and RemiseBB (Sabre)) is smaller. This is because, in Remise the client does not send the DPF keys, rather the servers construct the DPF from the index and the message shares. We observe that the validity check bandwidth(auditing + authorization) is independent of the message size for all systems. In Remise, the messages shares are only used once the authorization passes and they go to the access phase.

Request Validity Check Comparisons Figure 3a compares the wall-clock time for servers to *check the validity* of a single write request (includes DPF audit (if any), and authorization checks) in RemiseBB and Spectrum (PACL) under three network latencies: 10 ms, 30 ms, and 60 ms. We do not include a comparison with plain Spectrum, since integrating PACL improves its performance by more than an order of magnitude. For smaller database sizes, Spectrum (PACL) outperforms the *total* cost (preprocessing plus online) of RemiseBB. This behavior is expected. The preprocessing phase of RemiseBB —specifically, the MPC-based generation of shares of a standard-basis vector—requires $O(\log n)$ rounds of communication. In contrast, Spectrum (PACL) relies on the evaluation of a verifiable DPF, which, while computationally more expensive than a standard DPF, avoids this additional round complexity. As the database size increases, computational overhead begins to dominate latency-induced costs. This trend is reflected in our results: at lower latencies, the crossover point at which RemiseBB outperforms Spectrum (PACL) occurs at smaller database sizes. We were unable to execute the Spectrum (PACL) implementation beyond 2^{26}; however, extrapolating from existing measurements, the performance gap is expected to further favor RemiseBB at larger scales. Finally, when considering only the online phase, the gains are an order of magnitude larger. For example, at a database size of 2^{26}, RemiseBB provides nearly an $80\times$ speedup over Spectrum (PACL).

Figure 3b compares the time required to validate a request in RemiseBB (Sabre) against the time required to validate a single request in Express (PACL). Since RemiseBB (Sabre)'s implementation amortizes latency

[4]Code from https://github.com/sachaservan/pacl/tree/main

Table 2: Server-Side Cost Comparison of Remise Variants. Cost mentioned in gray is the preprocessing cost.

Operation	Local Computation	Bandwidth	Rounds
Remise Write	$O(n) + O(n)$	$O(\lg n) + O(1)$	$O(\lg n) + O(1)$
RemiseCC Read	$O(n) + O(n \cdot m)$	$O(\lg n) + O(m)$	$O(\lg n) + O(1)$
RemiseCC Update Authorization	$O(n) + O(n \cdot m)$	$O(\lg n) + O(m)$	$O(\lg n) + O(1)$

Table 3: Bandwidth (in bytes) decomposition across different systems during the write operation. We fix the database size at 2^{20}. In the table, n denotes the number of write requests, k out-of-n requests are authorized, m denotes the size of the payload. The number of LowMC rounds in RemiseBB (Sabre) is denoted as `rnds`. The DOROM operation in Remise uses LowMC cipher, thus it appears in the Req. validity Check bandwidth. The gray values represents the preprocessing phase communication. Spectrum and Express communications were calculated from their source code.

System	Client Req. Size writer $\to \mathtt{S}_b$	Req. Validity Check $\mathtt{S}_b \to \mathtt{S}_{1-b}$	Access $\mathtt{S}_b \to \mathtt{S}_{1-b}$
Remise	$(m+24) \cdot n$	$960 \cdot n + 16 \cdot (2 \cdot \mathtt{rnds} + 3) \cdot n$	$m \cdot k$
RemiseBB (Sabre)	$(m+24) \cdot n$	$16 \cdot (\mathtt{rnds} + 3)n$	$m \cdot k$
Spectrum (PACL)	$(m+3120) \cdot n$	$65 \cdot n$	0
Express (PACL)	$(m+376) \cdot n$	$16 \cdot n$	0

across a large batch of requests, while Express(PACL) is dominated by per request local computation, comparing these operating points makes the performance differences more apparent in the plots. We first point out that in RemiseBB (Sabre) there is no preprocessing phase for the authorization. Observe that since the authorization check in RemiseBB (Sabre) is PRF-based it is independent of the database size. In Express (PACL), however, since the authorization involves evaluating the DPF, the authorization time grows linearly with the database size. The upshot of this will be visible in the following paragraph in Figure 4b, where we will observe that the throughput for RemiseBB (Sabre) will be much higher than the throughput of Express (PACL).

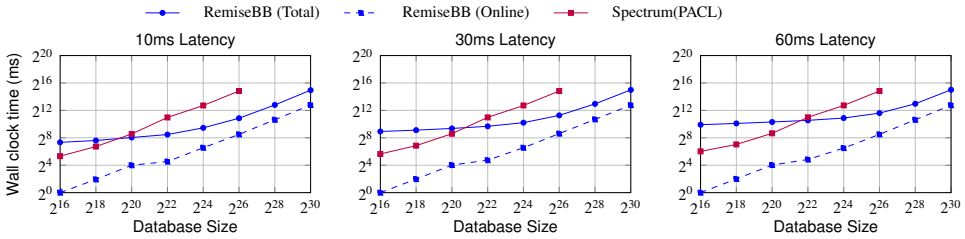
Throughput and Goodput Comparisons Figure 4 compares the throughput and goodput achieved by the different systems under varying fractions of unauthorized traffic. For RemiseBB (and RemiseBB (Sabre)), we evaluate two configurations: (i) using offline-preprocessed DPFs, and (ii) generating (on-the-fly) DPFs online during request processing.

Observe in Figure 4a that online DPF generation significantly reduces both throughput and goodput compared to the offline-preprocessed setting. In contrast, when DPFs are preprocessed offline, increasing the fraction of unauthorized traffic improves throughput and goodput. This is because unauthorized requests terminate after the lightweight authorization phase and avoid the more expensive access phase.

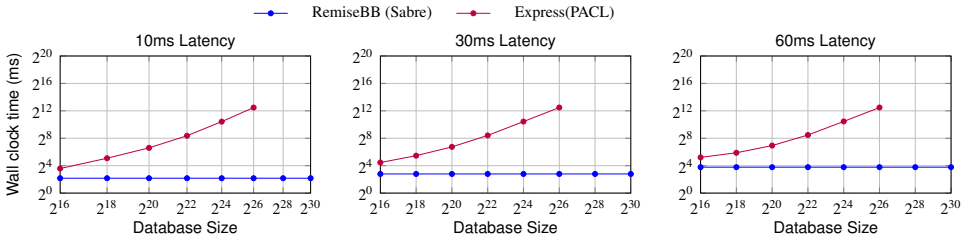
Finally, Figure 4b shows that as the fraction of unauthorized traffic increases, RemiseBB (Sabre) with on-the-fly DPF generation, surpasses Express (PACL) at progressively smaller database sizes. This is because the authorization cost of RemiseBB (Sabre) is $O(1)$, since it avoids DPF expansion during authorization.

Cost Breakdown and Effect of Message Size Figure 5 shows the breakdown of the total wall-clock time into preprocessing, online validity check, and access costs for different message sizes. We fix the size of the database to 2^{20} and network latency to 30 ms. We observe that the online validity check contributes only a small fraction of the total execution time in Remise. The preprocessing cost appears for RemiseBB and RemiseBB (Sabre), while the remaining systems do not perform any preprocessing. Furthermore, the access cost dominates the overall runtime and increases significantly with the message size. Observe that, the access costs for all systems are identical, as they rely on the same underlying DPF-based access mechanism. The upshot of this is that, as the message size increases, the throughput and goodput are similarly affected across systems. For all systems, the validity check cost remains nearly constant across different message sizes, since the validity of a request is independent of the transferred payload size. The figure also illustrates that unauthorized requests avoid the expensive memory-access phase entirely; consequently, the large "green" access component would be absent for such requests, reducing both wall-clock time and bandwidth consumption.

RemiseCC vs. PACL-Inclusion Predicate In Figure 6, we benchmark the time taken by the servers to perform authorization checks on read requests in RemiseCC. We compare against PACL's inclusion predicate access control, as it provides the same functionality of having multiple read authorization tokens per index. The experiment is done on a database of size 2^{24} while varying



(a) RemiseBB vs. Spectrum (PACL); experiments for Spectrum (PACL) were limited to input size 2^{26} due to infrastructure constraints. The corresponding error bars are small; for example, for RemiseBB at 30 ms latency and input size 2^{24}, the observed variation is ± 14.597 ms for total time and ± 8.8908 ms for online time.



(b) RemiseBB (Sabre) vs. Express (PACL); experiments for Express (PACL) were limited to size 2^{26} due to infrastructure constraints. The variance is too small, thus the error bars are not visible.

Figure 3: Wall-clock time comparison of request validity check during write operations in RemiseBB with PACL systems for different latencies while varying the database size.

the number access tokens per index. We do not plot the online time for RemiseCC as it is close to the total time because we use 2P-Duoram, which uses a Computationally symmetric PIR (CSPIR) [30].

6 Related Work

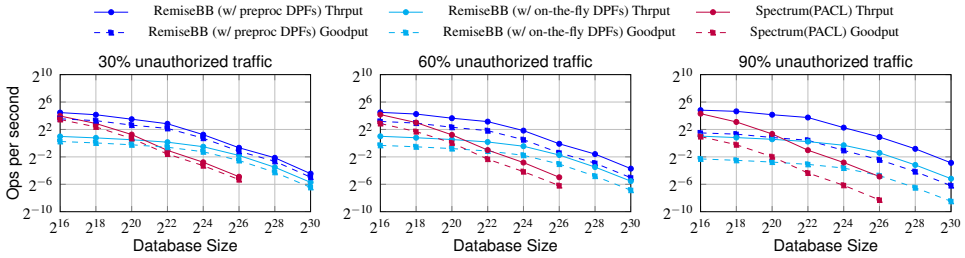
DC-nets based Systems: Anonymous communication systems often use dining cryptographer networks (DC-nets) to achieve sender-recipient unobservability, where one participant sends a real message while others send cover traffic. However, DC-nets suffer from high communication overhead, poor scalability, and disruption attacks, where a malicious participant can corrupt a broadcast using an invalid share. Dissent [8] introduces a costly blame protocol to identify disruptors but cannot prevent disruptions. Verdict [9] improves this using proofs of correctness, enabling verifiable DC-nets that prevent malformed contributions.

PIR-based Systems: Pynchon Gate [26] uses private information retrieval (PIR) for pseudonymous mail retrieval. Pung [1] extends this with a PIR-based key-value store where clients privately retrieve encrypted messages indexed by keys derived from shared secrets. Talek [6] supports private group messaging via a private log indexed using blocked cuckoo hashing [24], with clients using a PRF on a secret log handle to derive pseudo-random locations. Distributed-PIR [29] reduces server computation by offloading work to clients, increasing client overhead.

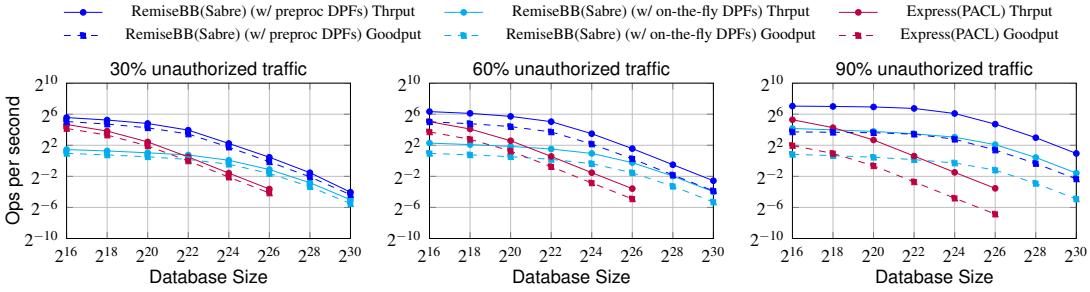
Mix-Net Based Systems: Mix-net systems obfuscate message sources through relays and shuffling. Riffle [17] uses verifiable shuffles and relies on the *anytrust* model, where anonymity holds if one server is honest. Cmix [5] and Trellis [20] improve efficiency using precomputation. Atom [16] iteratively shuffles messages across server groups, requiring one honest server per group. XRD [18] assigns clients mailboxes and uses loopback messages and shared chains to hide communication patterns.

Differential Privacy based Systems: Vuvuzela [32] pioneered differential privacy based anonymous messaging using shared drop locations derived from shared keys. Karaoke [21] improves Vuvuzela with efficient noise verification. Groove [2] further improves flexibility by allowing users to pre-submit messages, go offline without privacy loss, and communicate concurrently with multiple users.

ORAM-based Systems: Myco [15] introduces an ORAM-based metadata-private messaging system using an asymmetric two-server model, where servers emulate the ORAM client and server. Myco achieves polylogarithmic per-access server work, outperforming PIR and DPF-based systems. However, Myco does not provide server-enforced access control: any client can read any



(a) RemiseBB vs. Spectrum (PACL); experiments for Spectrum (PACL) were limited to database size 2^{26} due to infrastructure constraints. The corresponding error bars are small.



(b) RemiseBB (Sabre) vs. Express (PACL); All values for RemiseBB (Sabre) uses on-the-fly DPFs. Experiments for Express (PACL) were limited to database size 2^{26} due to infrastructure constraints. The corresponding error bars are small.

Figure 4: Throughput and Goodput Performance: We vary the database size and keep the latency fixed at 30ms. The message size in these experiments is 32 bytes.

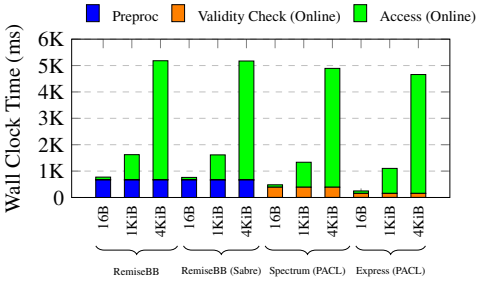


Figure 5: Breakdown of total cost for different message sizes for database size 2^{20} and 30 ms network latency. In the plot we cannot see the Validity Check (online) time for RemiseBB, since it is very small (around 15 ms).

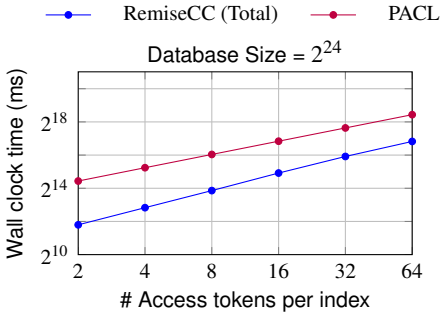


Figure 6: Comparing RemiseCC and PACL's read authorization. In these experiments we do not have a latency set between the servers since the local computation dominates the cost.

location and must trial-decrypt entries. While Remise presents a *Distributed*-ORAM based communication system, it focuses on improving access control in DPF-based systems by enabling servers to enforce authorized read and write locations and reject unauthorized accesses.

7 Conclusion

Remise enforces that only authorized clients can read from or write to the database. Importantly, the Remise servers perform access control checks without learning (i) the content of the messages written, (ii) which messages are being read, or (iii) any linkage between reads and writes.

Ethical Considerations

Stakeholder Identification and Impacts. We structure this section as a stakeholder-based analysis, distinguish-

ing between *direct stakeholders*—those who design, implement, or deploy Remise —and *indirect stakeholders*—those affected by systems built using Remise. Direct stakeholders include the research team and organizations that deploy anonymous communication services based on Remise. Indirect stakeholders include end users who rely on such systems for private and anonymous communication, journalists and whistleblowers who may depend on strong sender anonymity, regulators and auditors responsible for oversight, people who may be targeted by hateful rhetoric or online harassment by malicious actors, and adversaries who may attempt to disrupt or misuse anonymous communication infrastructures.

Research Team. This work involves no human subjects, no physical experiments, and no collection of real-world private data. All evaluation is conducted using synthetic workloads and controlled experimental settings. We explicitly acknowledge and respect the intellectual contributions of all authors.

Service Providers and System Deployers. Organizations deploying Remise (e.g., operators of anonymous bulletin boards or private communication platforms) may benefit from improved efficiency, fast auditing, and strong authorization privacy. However, deployment requires careful engineering and cryptographic review. Misconfiguration or implementation errors could lead to service disruption, denial of access for honest users, or weakened anonymity guarantees. To mitigate these risks, we document system assumptions, protocol tradeoffs, and performance characteristics in detail.

End Users and Society. *Beneficence.* Remise is designed to strengthen privacy and freedom of expression by enabling authorized anonymous communication without revealing access patterns, sender–receiver relationships, or authorization state. The system is particularly relevant for high-risk users such as journalists, whistleblowers, activists, and individuals operating under surveillance. *Respect for Persons.* We carefully cite prior work in anonymous communication, DPF-based systems, and DORAMs to ensure accurate attribution and to avoid misrepresentation of related techniques. The design of Remise explicitly avoids requiring users to reveal identities, long-term metadata, or sensitive personal information to the servers. *Justice.* The protocols underlying Remise rely only on standard cryptographic primitives and do not require specialized hardware or proprietary infrastructure. Our goal is to keep the core techniques accessible to researchers and system designers worldwide, reducing barriers to entry and enabling independent verification and reuse.

Respect for Law and Public Interest. We recognize that anonymous communication systems may raise concerns for regulators and law-enforcement agencies. Remise is designed to operate within the bounds of applicable law, assuming lawful deployment and governance by service providers. The system provides authorization controls that restrict which locations clients may read or write, balancing anonymous communication with server-enforced access control without requiring surveillance of honest users. Sometimes, malicious actors may conduct targeted harassment or spread hateful rhetoric. While these campaigns are not strictly *unlawful* they are nevertheless harmful, and a system like Remise could enable such abuse. Minority / non-normative populations are particularly at risk of this sort of abuse in online spaces.

Publication and Research Impact. This work contributes new techniques for combining DORAMs, authorization, and anonymous communication, and may influence future research on privacy-preserving systems. To reduce misuse or misinterpretation, we document key design decisions and tradeoffs, including the efficiency benefits of read-only and write-only memory instantiations, the assumption of at least one honest server, and the limitations of the threat model.

Potential Harms and Dual-Use Risks. We acknowledge that anonymous communication systems can be misused by malicious actors, including for spreading false or harmful information anonymously. Additionally, immature or unreviewed deployments of Remise could harm honest users by causing loss of availability or weakened anonymity guarantees. We therefore encourage responsible deployment, thorough security auditing, and appropriate legal and organizational oversight. We emphasize that Remise is a research prototype and that production deployments should be preceded by careful review.

Decision to Conduct and Publish. This research was initiated to improve the efficiency, robustness, and accountability of anonymous communication systems. We considered the dual-use nature of such systems and concluded that the benefits to privacy, free expression, and secure communication outweigh the potential risks when the system is deployed responsibly. The decision to publish followed internal review, correctness validation, and performance evaluation.

Open Science

Our code is available via permanent link here: https://zenodo.org/records/20438384. We provide all artifacts necessary to reproduce the evaluation, available at: **Source Code:** (i) Code to benchmark Remise, (ii) Distributed Point Function (DPF) primitives, (iii) Secret-sharing abstractions and MPC building blocks, and (iv) Asynchronous networking framework used for protocol execution. **Benchmarking Infrastructure:** (i) Scripts (and commands) used to reproduce numbers reported in Figures 3–6, (ii) Configuration file examples, so that anyone can write their config file and run

any custom experiment, and (iii) Network-latency configuration scripts used to emulate the evaluation settings reported in the paper. **Reported Metrics:** (i) Throughput and goodput, (ii) Evaluation, audit, and write latency, (iii) Communication bandwidth, and (iv) 95% confidence intervals.

Acknowledgments

The authors used generative AI-based tools to revise the text, improve flow and correct any typos, grammatical errors, and awkward phrasing. The authors thank Anusandhan National Research Foundation (ANRF) for their support. This research work was partially supported by a travel grant from Satish and Kamlesh Agarwal, Houston, TX, USA.

References

[1] Sebastian Angel and Srinath Setty. Unobservable Communication over Fully Untrusted Infrastructure. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, 2016.

[2] Ludovic Barman, Moshe Kol, David Lazar, Yossi Gilad, and Nickolai Zeldovich. Groove: Flexible Metadata-Private Messaging. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022.

[3] Donald Beaver. Efficient Multiparty Protocols Using Circuit Randomization. In *CRYPTO*, pages 420–432, 1991.

[4] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function Secret Sharing: Improvements and Extensions. In *Proceedings of the 23rd ACM Conference on Computer and Communications Security (CCS)*, pages 1292–1303. ACM, 2016.

[5] David Chaum, Debajyoti Das, Farid Javani, Aniket Kate, Anna Krasnova, Joeri de Ruiter, and Alan T. Sherman. cMix: Mixing with Minimal Real-Time Asymmetric Cryptographic Operations. Cryptology ePrint Archive, Paper 2016/008, 2016.

[6] Raymond Cheng, William Scott, Elisaweta Masserova, Irene Zhang, Vipul Goyal, Thomas Anderson, Arvind Krishnamurthy, and Bryan Parno. Talek: Private Group Messaging with Hidden Access Patterns. In *Proceedings of the 36th Annual Computer Security Applications Conference*, ACSAC '20, pages 84–99, 2020.

[7] Henry Corrigan-Gibbs, Dan Boneh, and David Mazières. Riposte: An Anonymous Messaging System Handling Millions of Users. In *2015 IEEE Symposium on Security and Privacy, SP 2015, San Jose, CA, USA, May 17-21, 2015*, pages 321–338. IEEE Computer Society, 2015.

[8] Henry Corrigan-Gibbs and Bryan Ford. Dissent: accountable anonymous group messaging. In Ehab Al-Shaer, Angelos D. Keromytis, and Vitaly Shmatikov, editors, *Proceedings of the 17th ACM Conference on Computer and Communications Security, CCS 2010, Chicago, Illinois, USA, October 4-8, 2010*, pages 340–350. ACM, 2010.

[9] Henry Corrigan-Gibbs, David Isaac Wolinsky, and Bryan Ford. Proactively Accountable Anonymous Messaging in Verdict. In Samuel T. King, editor, *Proceedings of the 22th USENIX Security Symposium, Washington, DC, USA, August 14-16, 2013*, pages 147–162. USENIX Association, 2013.

[10] Cécile Delerablée, Pascal Paillier, and David Pointcheval. Fully Collusion Secure Dynamic Broadcast Encryption with Constant-Size Ciphertexts or Decryption Keys. In *Pairing-Based Cryptography – Pairing 2007*, volume 4575 of *Lecture Notes in Computer Science*, pages 39–59. Springer, 2007.

[11] Jack Doerner and Abhi Shelat. Scaling ORAM for Secure Computation. In *CCS*, pages 523–535. ACM, 2017.

[12] Wenliang Du and Mikhail J. Atallah. Protocols for Secure Remote Database Access with Approximate Matching. In *E-Commerce Security and Privacy (Part II)*, Advances in Information Security, February 2001.

[13] Saba Eskandarian, Henry Corrigan-Gibbs, Matei Zaharia, and Dan Boneh. Express: Lowering the Cost of Metadata-hiding Communication with Cryptographic Privacy. In *30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021*, pages 1775–1792. USENIX Association, 2021.

[14] Freedom of the Press Foundation. SecureDrop, 2025. Open-source whistleblower submission system. Latest release: 2.12.8 (May 8, 2025).

[15] Darya Kaviani, Deevashwer Rathee, Bhargav Annem, and Raluca Ada Popa. Myco: Unlocking Polylogarithmic Accesses in Metadata-Private Messaging. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 4419–4437, 2025.

[16] Albert Kwon, Henry Corrigan-Gibbs, Srinivas Devadas, and Bryan Ford. Atom: Horizontally Scaling Strong Anonymity. In *Proceedings of the 33rd ACM Symposium on Operating Systems Principles (SOSP)*, pages 463–480. ACM, 2017.

[17] Albert Kwon, David Lazar, Srinivas Devadas, and Bryan Ford. Riffle: An Efficient Communication System With Strong Anonymity. *Proc. Priv. Enhancing Technol.*, 2016(2), 2016.

[18] Albert Kwon, David Lu, and Srinivas Devadas. XRD: Scalable Messaging System with Cryptographic Privacy. In Ranjita Bhagwan and George Porter, editors, *17th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2020, Santa Clara, CA, USA, February 25-27, 2020*, pages 759–776. USENIX Association, 2020.

[19] Emily B. Laidlaw. *Regulating Speech in Cyberspace: Gatekeepers, Human Rights and Corporate Responsibility*. Cambridge University Press, 2015.

[20] Simon Langowski, Sacha Servan-Schreiber, and Srinivas Devadas. Trellis: Robust and Scalable Metadata-private Anonymous Broadcast. Cryptology ePrint Archive, Paper 2022/1548, 2022.

[21] David Lazar, Yossi Gilad, and Nickolai Zeldovich. Karaoke: Distributed Private Messaging Immune to Passive Traffic Analysis. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 711–725, Carlsbad, CA, October 2018. USENIX Association.

[22] Zachary Newman, Sacha Servan-Schreiber, and Srinivas Devadas. Spectrum: High-bandwidth Anonymous Broadcast. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, 2022.

[23] Jason R. C. Nurse. Cybercrime and You: How Criminals Attack and the Human Factors That They Seek to Exploit. *The Oxford Handbook of Cyberpsychology*, pages 662–690, October 2018.

[24] Rasmus Pagh and Flemming Friche Rodler. Cuckoo hashing. *Journal of Algorithms*, 51(2):122–144, 2004.

[25] Rohan Ravi, Paritosh Shukla, and Adithya Vadapalli. Remise: Authorized anonymous communication systems. Cryptology ePrint Archive, Paper 2026/481, 2026.

[26] Len Sassaman, Bram Cohen, and Nick Mathewson. The pynchon gate: a secure method of pseudonymous mail retrieval. In *Proceedings of the 2005 ACM Workshop on Privacy in the Electronic Society*, WPES '05, pages 1–9, New York, NY, USA, 2005. Association for Computing Machinery.

[27] Sajin Sasy and Ian Goldberg. SoK: Metadata-Protecting Communication Systems. *Proc. Priv. Enhancing Technol.*, 2024(1):509–524, 2024.

[28] Sacha Servan-Schreiber, Simon Beyzerov, Eli Yablon, and Hyojae Park. Private Access Control for Function Secret Sharing. In *2023 IEEE Symposium on Security and Privacy (SP)*, 2023.

[29] Elkana Tovey, Jonathan Weiss, and Yossi Gilad. Distributed PIR: Scaling Private Messaging via the Users' Machines. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, CCS '24, pages 1967–1981, New York, NY, USA, 2024. Association for Computing Machinery.

[30] Adithya Vadapalli, Ryan Henry, and Ian Goldberg. Duoram: A Bandwidth-Efficient Distributed ORAM for 2- and 3-Party Computation. Cryptology ePrint Archive, Paper 2022/1747, 2022. https://eprint.iacr.org/2022/1747.

[31] Adithya Vadapalli, Kyle Storrier, and Ryan Henry. Sabre: Sender-Anonymous Messaging with Fast Audits. In *IEEE Symposium on Security and Privacy (SP)*, pages 1953–1970, 2022.

[32] Jelle van den Hooff, David Lazar, Matei Zaharia, and Nickolai Zeldovich. Vuvuzela: scalable private messaging resistant to traffic analysis. In Ethan L. Miller and Steven Hand, editors, *Proceedings of the 25th Symposium on Operating Systems Principles, SOSP 2015, Monterey, CA, USA, October 4-7, 2015*, pages 137–152. ACM, 2015.